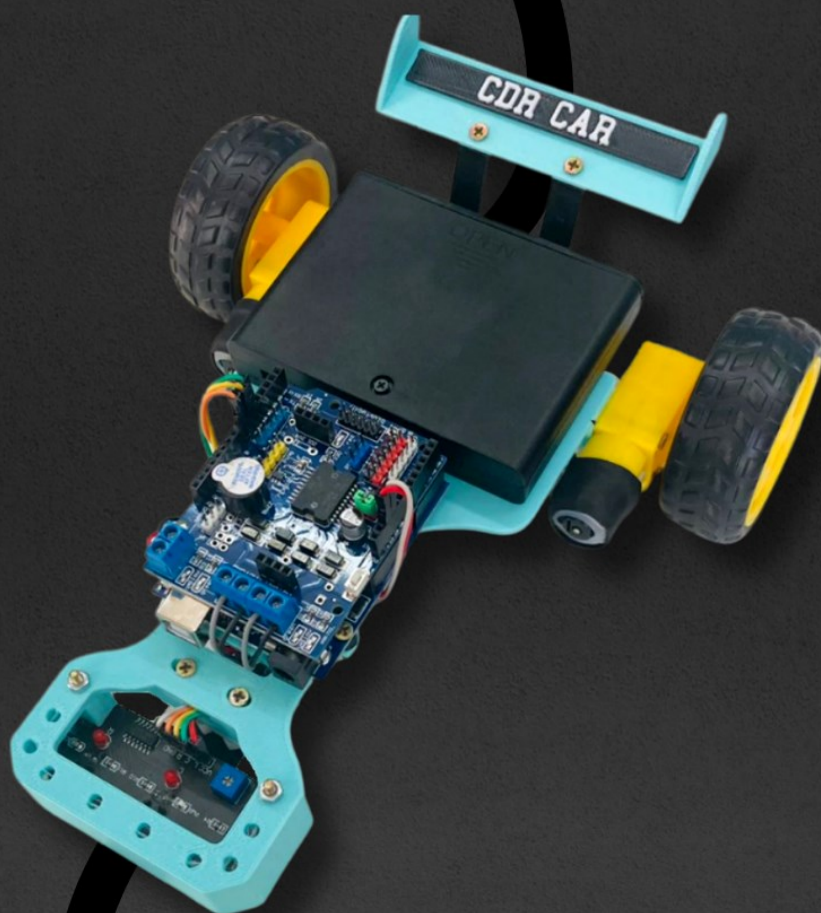
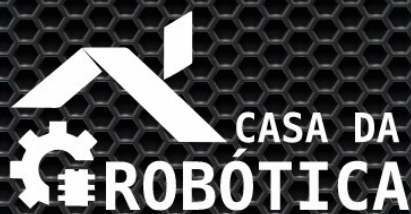


CDR CAR

ROBÔ SEGUIDOR
DE LINHA



MATERIAL DE APOIO



Conselho Editorial

Luan Silva Santana

Wellington Assunção Azevedo

Diagramação e Capa

Carol Correia Viana

Produção e Revisão

Carol Correia Viana

Kleber Rocha Bastos

Colaborador

Carlos Dyorgenes Santana

Modelagem 3D do CDR CAR

Kleber Rocha Bastos

Contato

contato@casadarobotica.com

 [@casadaroboticaoficial](https://www.facebook.com/casadaroboticaoficial)

 [@casadarobotica](https://www.instagram.com/casadarobotica)

www.casadarobotica.com



Copyright © 2022 da WL Componentes Eletrônicos Ltda.

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito dos autores.

Editores: Luan Silva Santana e Welligton Assunção Azevedo

Diagramação e capa: Carol Correia Viana

Produção e revisão: Carol Correia Viana e Kleber Rocha Bastos

Colaboração: Carlos Dyorgenes Santana

Primeira Edição

WL Componentes Eletrônicos

CNPJ: 29.495.665/0001-03

Vitória da Conquista, BA - Brasil

Tel.: +55 77 99151-2820

E-mail: contato@casadarobotica.com

Site: www.casadarobotica.com

APRESENTAÇÃO

Olá,

Obrigada por adquirir o kit CDR CAR: robô seguidor de linha.

Este material constitui um guia de apoio para seus estudos com o CDR car. Nele, preparamos uma série de conteúdos importantes para seus primeiros passos com o kit.

Inicialmente, apresentamos as etapas passo a passo para a montagem mecânica do CDR CAR.

Em seguida, discorremos sobre a plataforma Arduino e a placa microcontroladora UNO SMD R3 Atmega328, compatível com ao projeto Arduino UNO, ressaltando suas principais características e forma de programação através do software Arduino IDE.

Posteriormente, foram expostas as principais características e especificações dos demais componentes eletrônicos do Kit. Neste momento, o esquemático e montagem eletrônica do CDR CAR foi explicado minuciosamente.

Logo após, apresentamos algumas noções da linguagem de programação para microcontroladores do projeto Arduino, desde a declaração de variáveis até o uso de bibliotecas.

A seguir, exibimos uma série de exemplos práticos para testar o funcionamento da Placa UNO, do Shield Ponte H L298P, dos motores e do módulo seguidor de linha.

Por fim, para que você aprenda se divertindo, propomos o uso do CDR CAR como um robô seguidor de linha, detalhando a lógica e programação necessária.

Esperamos que você aproveite esse material com entusiasmo e ele auxilie a sua jornada de estudos.

Um grande abraço,

Equipe Casa da Robótica

GRUPO DE SUPORTE NO TELEGRAM

Escaneie o QR-CODE e faça parte do nosso grupo:



Ou acesse o endereço:

www.blogdarobotica.com/tel-cdr-car

Suporte técnico • Apoio ao estudo • Tira dúvidas
Troca de conhecimento • Atualização dos conteúdos
Materiais extras • Desafios

SUMÁRIO

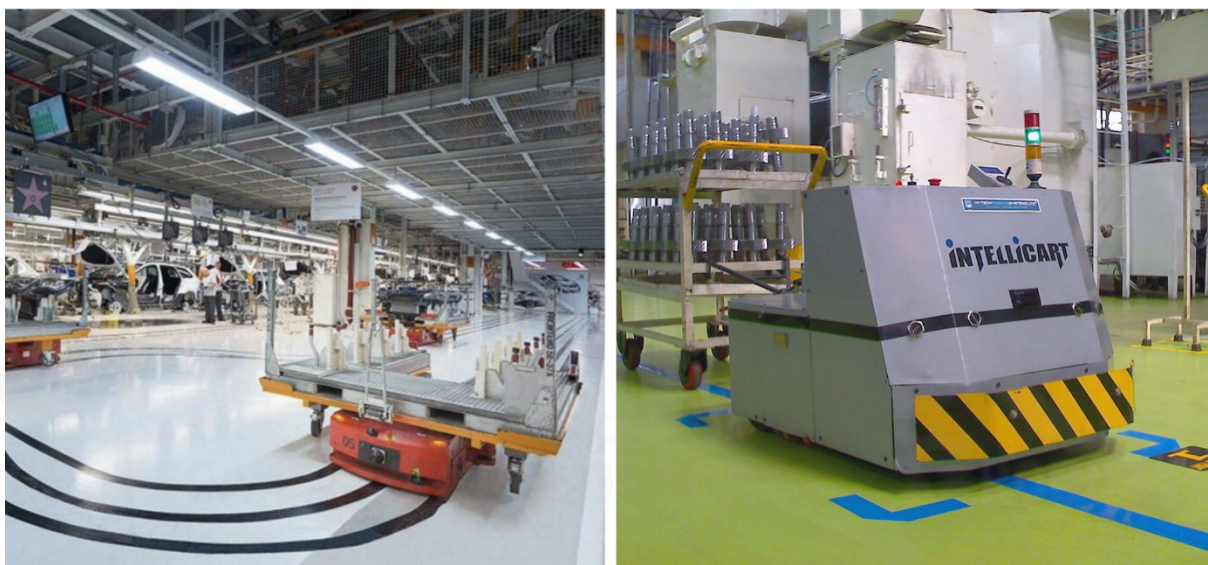
1.	INTRODUÇÃO	8
2.	MONTAGEM MECÂNICA	11
	PASSO A PASSO PARA MONTAGEM MECÂNICA DO CDR CAR	11
3.	MONTAGEM ELETRÔNICA	17
	CONHECENDO A PLATAFORMA ARDUINO	17
	O QUE É O ARDUINO?	17
	EXPLORANDO UMA PLACA UNO SMD R3 ATMEGA328	19
	PRIMEIROS PASSOS COM A PLACA UNO	22
	EXPLORANDO O ARDUINO IDE	23
	CONHECENDO OS DEMAIS COMPONENTES	29
	O QUE É O MOTOR DC?	29
	O QUE É O SHIELD PONTE H L298P?	32
	O QUE O MÓDULO SEGUIDOR DE LINHA COM SENSOR INFRAVERMELHO TCRT5000?	37
	O QUE É O SUPORTE PARA 6 PILHAS AA?	39
	O QUE SÃO OS JUMPERS?	40
	ESQUEMÁTICO ELÉTRICO DO CDR CAR	41
	PASSO A PASSO PARA MONTAGEM ELETRÔNICA DO CDR CAR	43
4.	FUNDAMENTOS DE PROGRAMAÇÃO	46
	PROGRAMAÇÃO PARA ARDUINO	46
	ESTRUTURAS BÁSICA DE UM SKETCH	47
	VARIÁVEIS	47
	FUNÇÕES	49
	BIBLIOTECAS	52
	OPERADORES	53
	OPERADORES ARITMÉTICOS	53
	OPERADORES DE COMPARAÇÃO E BOOLEANOS	53
	OPERADORES DE INCREMENTO E DECREMENTO	56
	ESTRUTURA DE CONTROLE DE FLUXO	56
	IF	57
	IF...ELSE	58
	FOR	58
	SWITCH...CASE	59
	WHILE	60
	DO...WHILE	60

VAMOS PROGRAMAR!	62
ALERTA SONORO! TESTANDO O BUZZER	62
MOVIMENTOS INICIAIS! TESTANDO OS MOTORES	66
MOVENDO O ROBÔ PARA FRENTE	66
MOVENDO O ROBÔ PARA TRÁS	70
DETECTANDO A LINHA! TESTANDO O MÓDULO SEGUIDOR DE LINHA	72
5. ROBÔ SEGUIDOR DE LINHA	79
CDR CAR: SEGUIDOR DE LINHA	79
ELABORANDO O CÓDIGO	81
REALIZANDO O UPLOAD DO CÓDIGO	89
CRIANDO A PISTA	89
DESAFIOS!	89
6. CONSIDERAÇÕES FINAIS	91
7. REFERÊNCIAS	92

1. INTRODUÇÃO

Robôs seguidores de linha são robôs pré-programados para detectar e seguir uma linha. Assim, a linha é o caminho em que o robô deve percorrer. Também conhecido como seguidor de faixa, esse tipo de robô é muito utilizado atualmente em indústrias e fábricas para transportar os materiais e caixas.

Figura 1 - Robôs seguidores de linha industrial.



Fonte: Geprom (2022) e Packt (2017).

O funcionamento de um robô seguidor de linha é relativamente simples. Ele deve seguir andando uma linha de cor preta ou branca, geralmente. A identificação da cor da linha é feita usando sensores, dispositivos capazes de detectar uma mudança do ambiente.

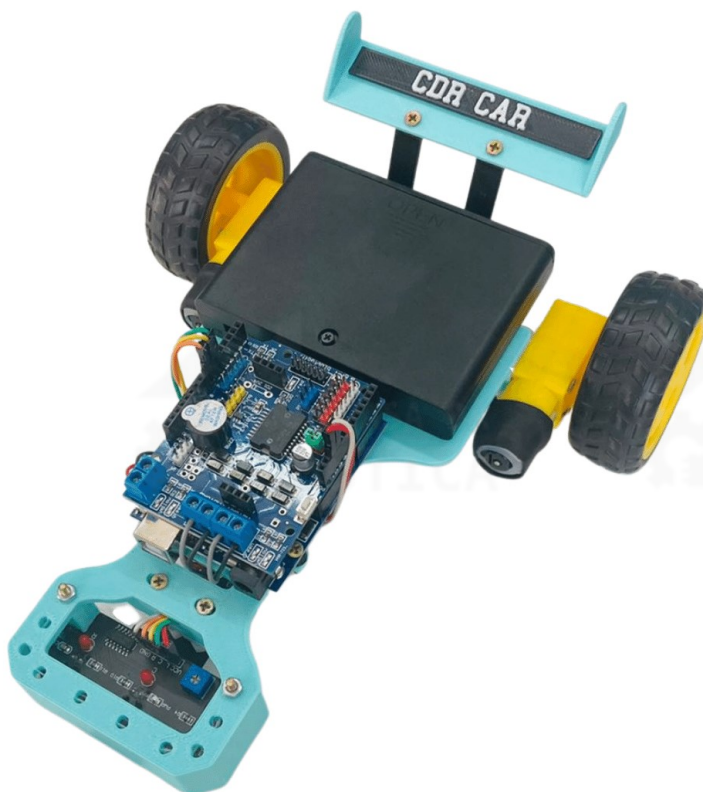
No universo *maker*, a construção de um robô seguidor de linha é um excelente projeto para quem deseja aprender eletrônica e programação na prática. O Kit CDR CAR foi pensado e desenvolvido para auxiliar a construção do seu primeiro robô seguidor de linha.

Durante a montagem mecânica do CDR CAR (Figura 2), você estimulará seu espírito maker. As peças do chassi do CDR CAR foram modeladas e produzidas em nosso laboratório de impressão 3D, com o intuito de facilitar a sua montagem. Cada

carrinho foi feito de forma única, ou seja, nenhum deles é exatamente igual ao outro.

As peças em 3D podem possuir algumas pequenas rebarbas, marcas de linhas entre camadas ou alguns fiapinhos que são característicos deste tipo de fabricação. Mas não se preocupe nada disso impedirá o funcionamento do seu CDR CAR! Nós fazemos todo o possível para te entregar um produto de alta qualidade.

Figura 2 - Robô seguidor de linha CDR CAR.



Com auxílio deste material de apoio, você aprenderá sobre a plataforma Arduino e a placa microcontroladora UNO SMD R3 Atmega328, compatível com o projeto Arduino UNO, e os demais componentes eletrônicos que o compõe.

A montagem eletrônica foi detalhada passo a passo e todas as partes eletrônicas necessárias já vão prontas para o uso sem a necessidade de realização de soldagem.

Preparamos o Capítulo 4 deste material de apoio para auxiliar você com os principais fundamentos da linguagem de programação utilizada pelo Arduino,

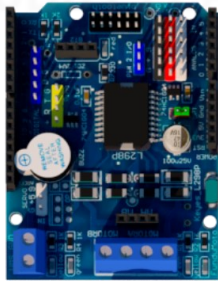
como declaração e tipos de variáveis, estrutura de condição e repetição, funções e biblioteca. Este capítulo também expõe uma série de exemplos práticos para que você teste o funcionamento da Placa UNO, do Shield Ponte H L298P, dos motores e do módulo seguidor de linha.

Por fim, para reunir os conhecimentos adquiridos até aqui, propomos o uso do CDR CAR como robô seguidor de linha. Toda a lógica e programação necessária foram detalhadas passo a passo.



2. MONTAGEM MECÂNICA

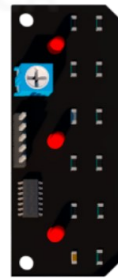
PASSO A PASSO PARA MONTAGEM MECÂNICA DO CDR CAR



Shield Ponte H



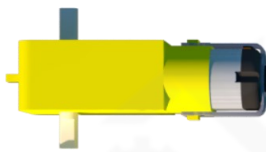
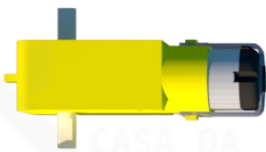
Placa Uno



Sensor de Linha



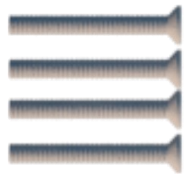
Rodas



Motores



Suportes do Aerofólio



M3x30 - 4pcs



M3x22 - 2pcs



Aerofólio



M3x12 - 6pcs



M3x20 - 6pcs



Case de Pilhas



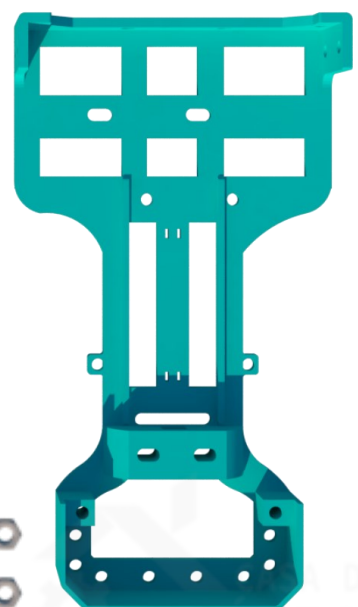
Apoio Placa Uno



Roda Boba



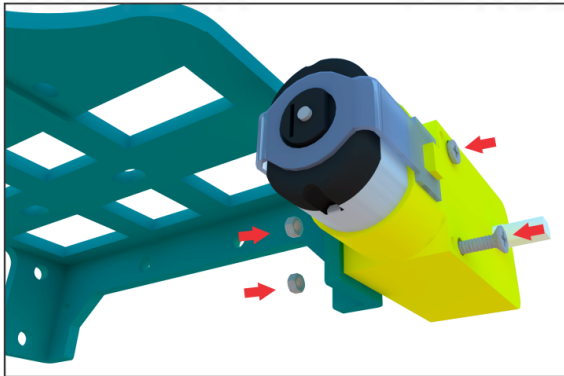
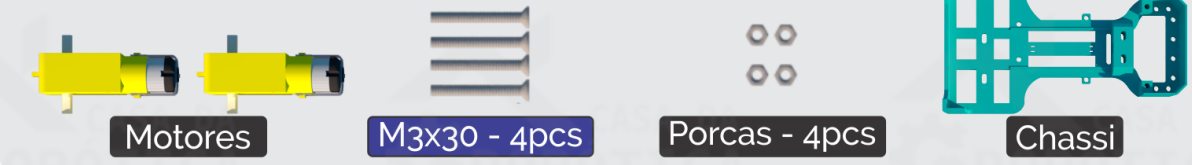
Porcas - 18pcs



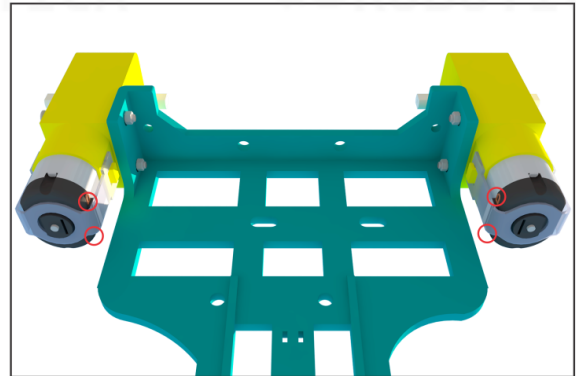
Chassi

Etapa 1

Itens necessários:



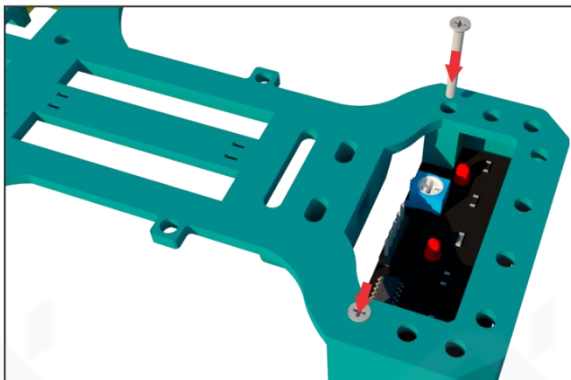
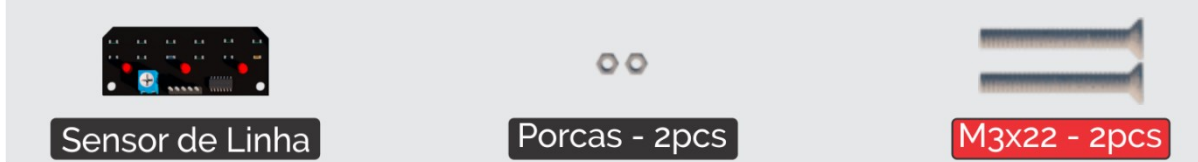
Fixe os 2 motores usando os parafusos M3x30 e porcas.



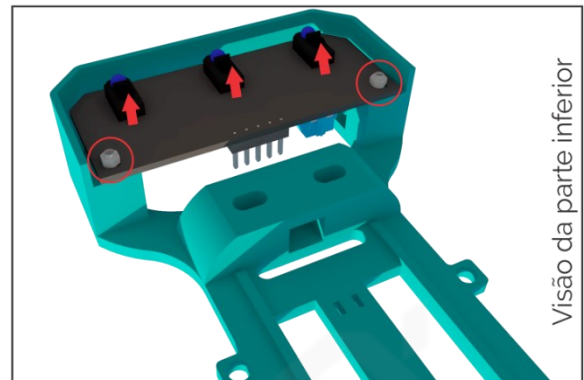
ATENÇÃO: Os fios dos motores devem ficar voltados para dentro.

Etapa 2

Itens necessários:



Prenda o sensor usando parafusos M3x25 e porcas.



Os sensores (LEDs azuis e pretos) devem ficar voltados para baixo.

Etapa 3

Itens necessários:



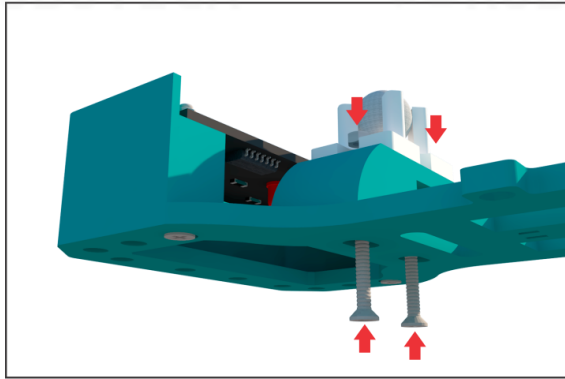
Roda Boba



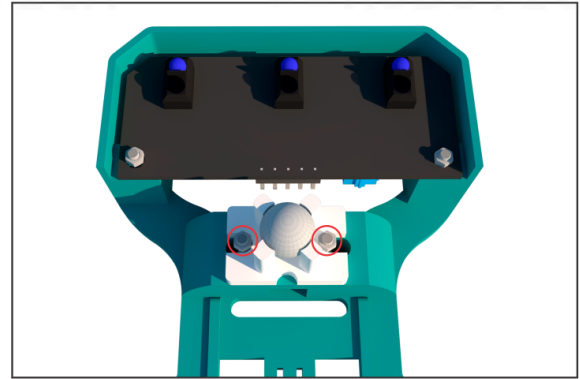
Porcas - 2pcs



M3x20 - 2pcs



Prenda a roda boba usando parafusos M3x20 e porcas.



Centralize a roda boba e aperte as porcas.

Etapa 4

Itens necessários:



Apoio Placa Uno



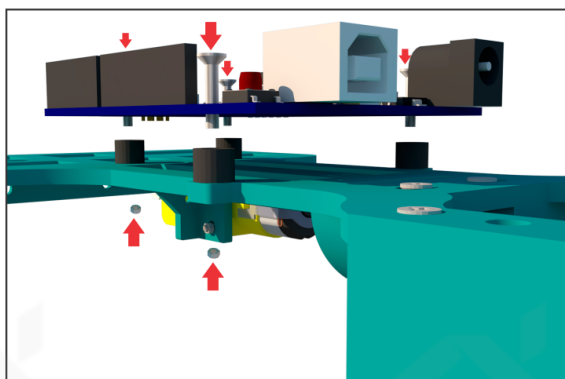
Placa Uno



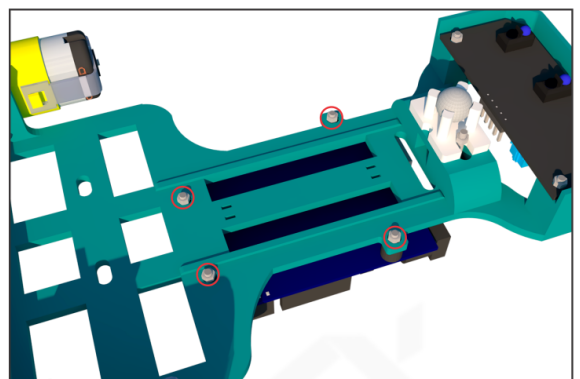
Porcas - 4pcs



M3x20 - 4pcs



Posicione os apoios para ficarem entre a placa UNO e o Chassi.



Fixe o conjunto com parafusos M3x20 e porcas.

Etapa 5

Itens necessários:



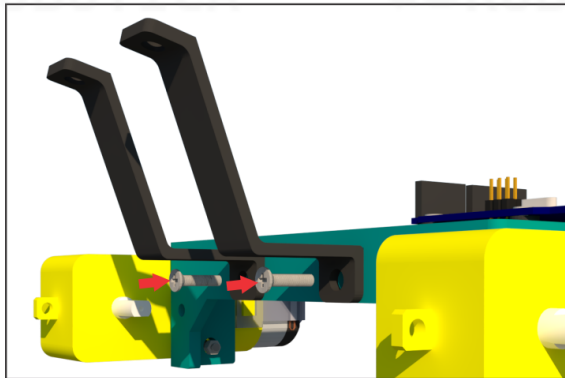
M3x12 - 2pcs



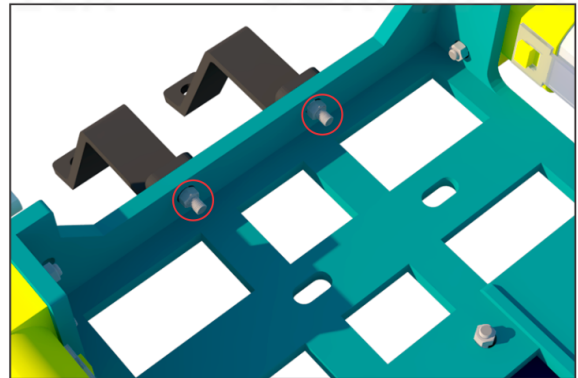
Porcas - 2pcs



Suportes do Aerofólio



Prenda os suportes do aerofólio com parafusos M3x12 e porcas.



NÃO aperte muito as porcas para não quebrar o suporte.

Etapa 6

Itens necessários:



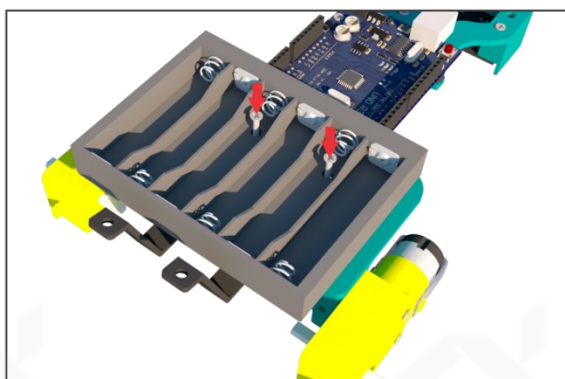
M3x12 - 2pcs



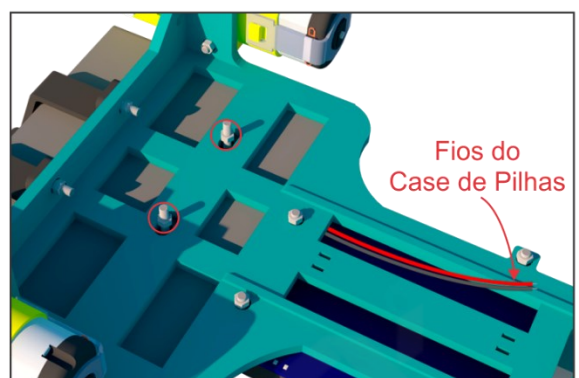
Porcas - 2pcs



Case de Pilhas



Prenda com parafusos M3x12, mas antes passe os fios por baixo da Placa UNO..



Depois de passado os fios, fixe os parafusos com as porcas.

Etapa 7

Itens necessários:



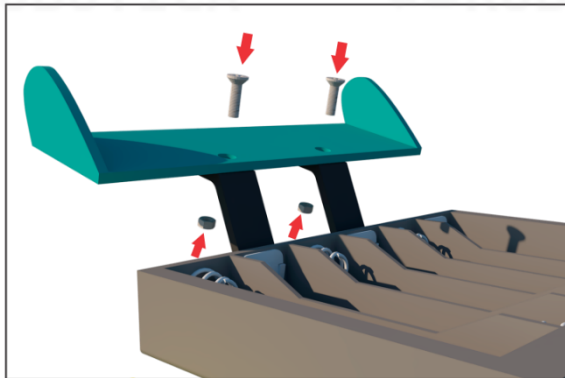
M3x12 - 2pcs



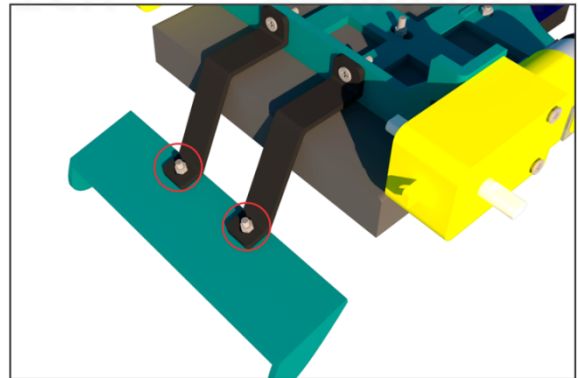
Porcas - 2pcs



Aerofólio



Fixe o aerofólio ao suporte usando parafusos M3x12.



NÃO aperte muito as porcas para não quebrar o suporte.

Etapa 8

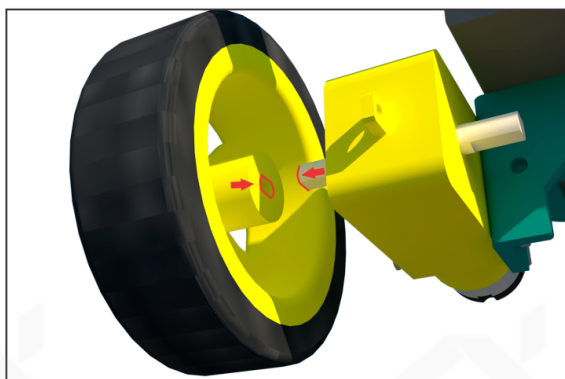
Itens necessários:



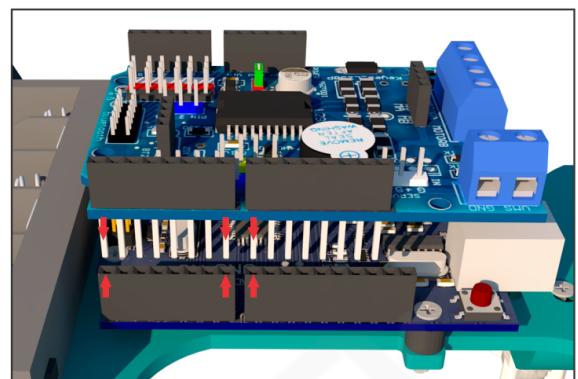
Rodas



Shield Ponte H



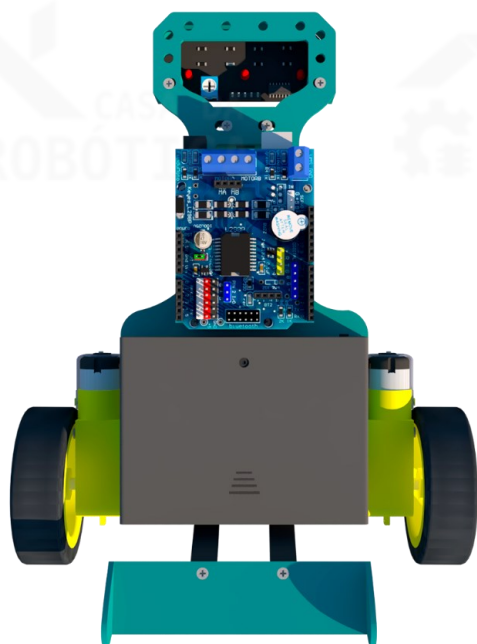
Encaixe as rodas nos motores de acordo com a posição do eixo.



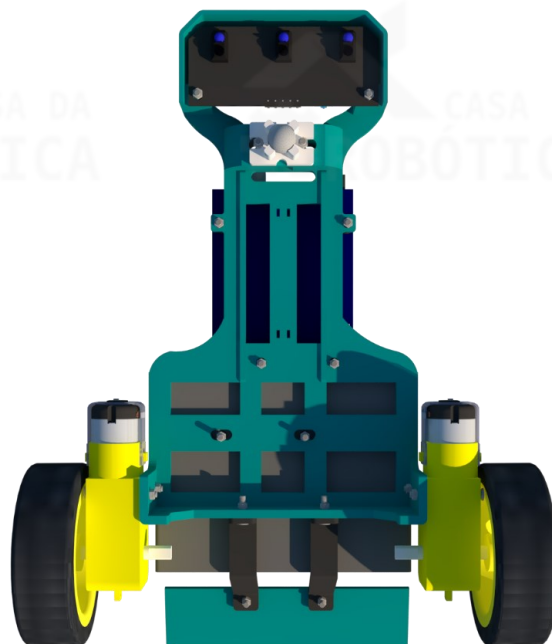
Encaixe o Shield na placa UNO de acordo com a posição acima.

Montagem concluída

Vista Superior



Vista Inferior



 **ATENÇÃO:** Não puxe, levante ou transporte o CDR CAR segurando-o pelo aerofólio, por ser uma peça **FRÁGIL**.

3. MONTAGEM ELETRÔNICA

CONHECENDO A PLATAFORMA ARDUINO

O QUE É O ARDUINO?

O Arduino é uma plataforma eletrônica de código aberto baseada em software e hardware de fácil utilização, sendo ideal para iniciantes e para qualquer pessoa que deseja construir projetos eletrônicos.

As placas Arduino permitem a conexão de circuitos eletrônicos aos seus terminais, o que possibilita a leitura de entradas – luz em um sensor, o acionamento de um botão ou uma mensagem SMS, e transformar estas informações em uma saída controlando algum dispositivo – por exemplo ligando um LED, ativando um motor ou enviando uma mensagem.

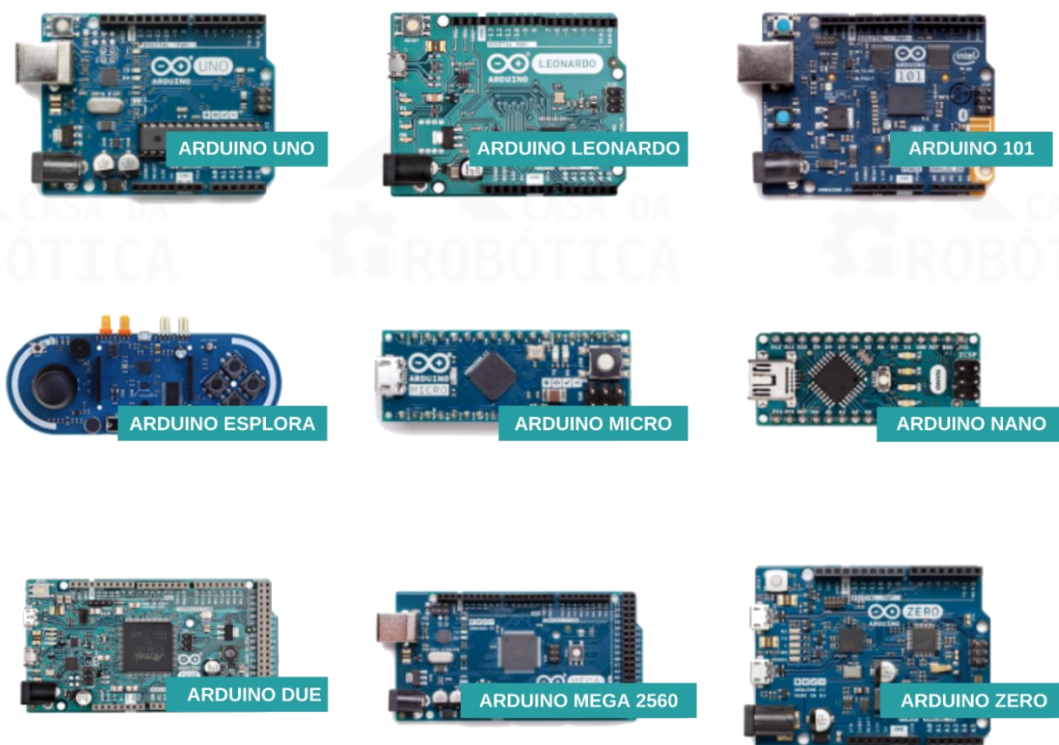
As placas Arduino podem ser conectadas ao computador por meio do barramento serial universal (USB), possibilitando sua utilização como placa de interface e controlar dispositivos por meio do seu computador.

A plataforma Arduino oferece uma série de vantagens em relação a outras plataformas, o que o tornou popular entre professores, alunos, amadores e projetistas, tais como:

- Possuir ambiente multiplataforma, ou seja, pode ser executado nos principais sistemas operacionais comercializáveis;
- Contar uma IDE de programação própria;
- Poder ser programado utilizando um cabo USB;
- Possuir hardware e software de fonte aberta;
- Ter sido desenvolvido em um ambiente educacional, sendo ideal para iniciantes.

Diante da sua popularização, a plataforma Arduino cresceu e atualmente conta com diversas versões de mercado. A Figura 3 ilustra algumas versões da placa Arduino.

Figura 3 - Algumas versões da placa Arduino



Além disso, existem uma série de placas compatíveis com o projeto Arduino, uma vez que seu hardware é aberto a replica destas placas são permitidas e possuem as mesmas características, pinagens e forma de uso. A Placa UNO SMD R3 Atmega328 disponível neste Kit, por exemplo, é compatível ao projeto do Arduino UNO.

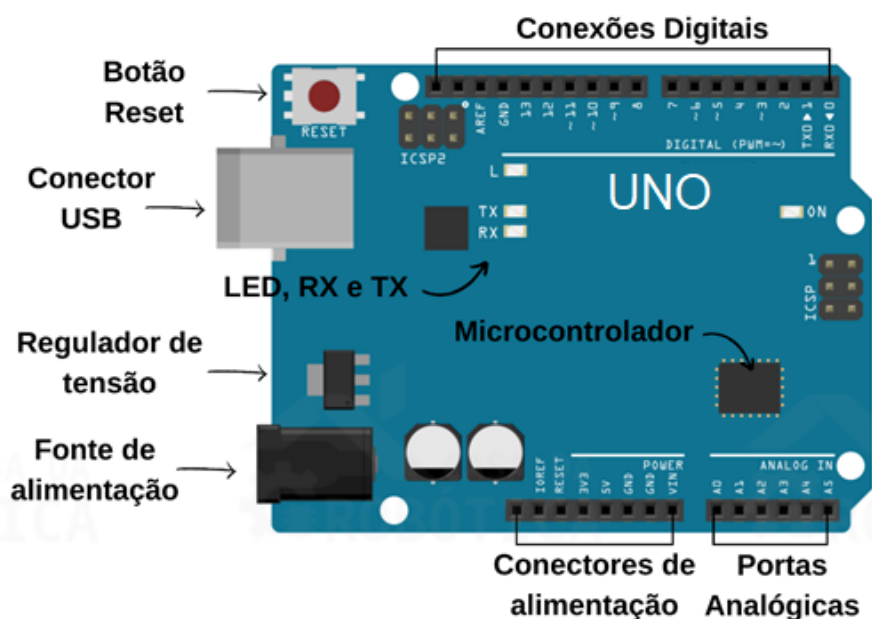
Existem placas Arduino bem pequenas (Nano, micro, mini), de tamanho médio e tradicional (Uno, Duemilanove, Leonardo), e as placas de maiores dimensões (Mega, Due). Diante de tanta variedade, você deve estar se perguntando: Qual placa devo usar no meu projeto? A escolha da versão ideal vai depender das necessidades de seu projeto, mas recomendamos:

- Placas de Arduino pequenas para projetos que precisam ser leves e ocupar pouco espaço;
- Placas de Arduino tamanho médio e tradicional para projetos de tamanho padrão como robôs, interfaces homem-máquina, central de monitoramento, entre outros;
- Placas de Arduino maiores dimensões para projetos que demandem de maior memória e número de portas de entrada e saída.

EXPLORANDO UMA PLACA UNO SMD R3 ATMEGA328

Conhecer os elementos que compõe a placa UNO SMD R3 ATMEGA328 é de suma importância antes de iniciar os nossos projetos. Desta forma, vamos explorar esta placa microcontroladora (Figura 4) para nos familiarizar com seus vários componentes.

Figura 4 - Placa microcontroladora UNO SMD R3 ATMEGA238.



Fonte de alimentação

O circuito interno da placa UNO deve ser alimentado com uma tensão contínua de 5V. Você pode alimentá-lo conectando-o a uma porta USB do computador, que fornecerá a alimentação e, também, a comunicação de dados, ou por meio de uma fonte de alimentação externa, que forneça uma saída contínua entre 7 V e 12 V, por meio da utilização de um plug P4 ou o pino Vin.

Regulador de tensão

Na placa UNO, o regulador de tensão tem como finalidade transformar qualquer tensão (entre 7 V e 12 V) que esteja sendo fornecida pelo conector de alimentação externa em uma tensão contínua de 5V.

Conectores de alimentação elétrica

Os conectores de alimentação elétrica fornecem energia para dispositivos externos e são constituídos pelos pinos:

- Reset que possui a mesma função do botão Reset;
- 3.3 V e 5 V que fornecem tensão de 3,3 e 5 V, respectivamente;
- GND fornece potencial de terra aos dispositivos externos;
- Vin fornece ao dispositivo externo a mesma tensão que está sendo recebida pelo pino de alimentação externa.

Entradas analógicas

A placa UNO possui 6 portas analógicas que estão indicados como Analog In, de A0 a A5. Esses pinos são dedicados a receber valores de grandezas analógicas, por exemplo, a tensão de um sensor. As grandezas analógicas variam continuamente no tempo dentro de uma faixa de valores.

Conexões digitais

A placa UNO possui 14 portas digitais que estão indicados como Digital, de 0 a 13. Estas portas podem ser utilizadas como receber ou enviar dados de grandezas digitais. Ao contrário das grandezas analógicas, as grandezas digitais não variam continuamente no tempo, mas sim em saltos entre valores definidos (0 ou 1, ligado ou desligado, sim ou não, 0 V ou 5 V).

Estes pinos digitais operam em 5V e corrente máxima de 40 mA. Além disso, alguns deles possuem funções especiais, como:

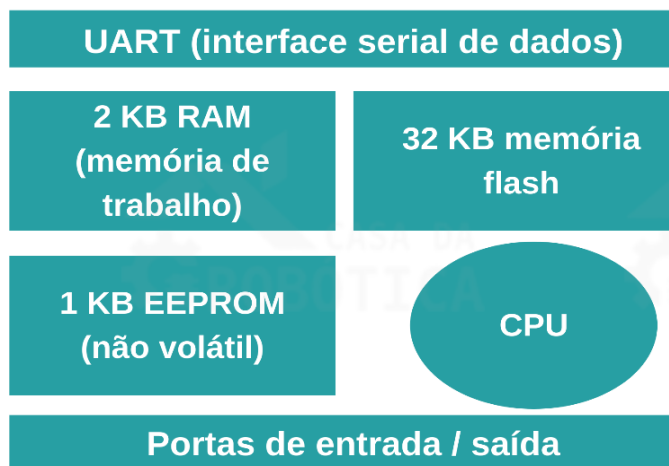
- Pinos 3, 5, 6, 9, 10 e 11 podem ser usados como saídas PWM, simulando uma porta analógica;
- Pinos 0 e 1 (RX e TX) podem ser utilizados para comunicação serial;
- Pinos 2 e 3 podem ser configurados para gerar uma interrupção externa.

Microcontrolador

O microcontrolador utilizado na placa UNO é o ATmega328, um pequeno chip de 28 pinos que se encontra no centro da placa e é considerado o cérebro deste dispositivo. Este único chip é um pequeno computador, contendo memória, processador e toda eletrônica necessária aos pinos de entrada e saída.

É no microcontrolador que tudo acontece, é nele que fica gravado o código desenvolvido para execução. O microcontrolador permite que a placa UNO funcione de forma autônoma, em outras palavras, uma vez transferido o código não existe mais a necessidade de comunicação com o computador. Um fato que deve ser lembrado é que ao gravar um código, o anterior é descartado, ficando apenas o último código gravado. Algumas características do microcontrolador ATmega328 encontra-se detalhado na Figura 5 abaixo.

Figura 5 - Diagrama de blocos do ATmega328.



Fonte: Adaptado de Monk (2017).

Botão Reset

O botão Reset tem como única função reinicializar a placa microcontroladora.

Outros componentes

Além dos componentes citados, a placa UNO também conta com um oscilador a cristal, capaz de realizar 16 milhões de ciclos ou oscilações por segundo, conector serial de programação, outro meio de programar a placa UNO, e um chip de interface USB, que converte os níveis de sinal usados pelo padrão USB em níveis que podem ser usados pela placa UNO.

PRIMEIROS PASSOS COM A PLACA UNO

Para que a placa UNO execute qualquer ação, você precisará escrever um código ou Sketch em linguagem C/C++ utilizando gratuitamente o software Arduino IDE, que se encontra disponível na versão online e offline, e depois fazer o upload deles para a placa.

O Arduino Web Editor é a interface de desenvolvimento online do Arduino, com ele é possível codificar, salvar os esboços na nuvem, fazer backup e enviar o código feito para qualquer placa compatível com o Arduino a partir do navegador de internet. Por estar hospedado online, o Arduino Web Editor estará sempre atualizado com os recursos, bibliotecas e suporte mais recente. Além disto, esta interface de desenvolvimento permite que você acesse um código salvo a partir de qualquer dispositivo conectado à internet. O Arduino Web Editor encontra-se disponível no link <https://create.arduino.cc/editor>, em que será necessário a realização de um cadastro de acesso.

O Arduino IDE é a versão offline desta ferramenta de desenvolvimento e pode ser executado no Windows, Mac OS X e Linux. O download do Arduino IDE encontra-se disponível no link <https://www.arduino.cc/en/Main/Software#download>, em que se faz necessária a escolha da versão apropriada para seu sistema operacional.

Download e Instalação do Arduino IDE no Windows

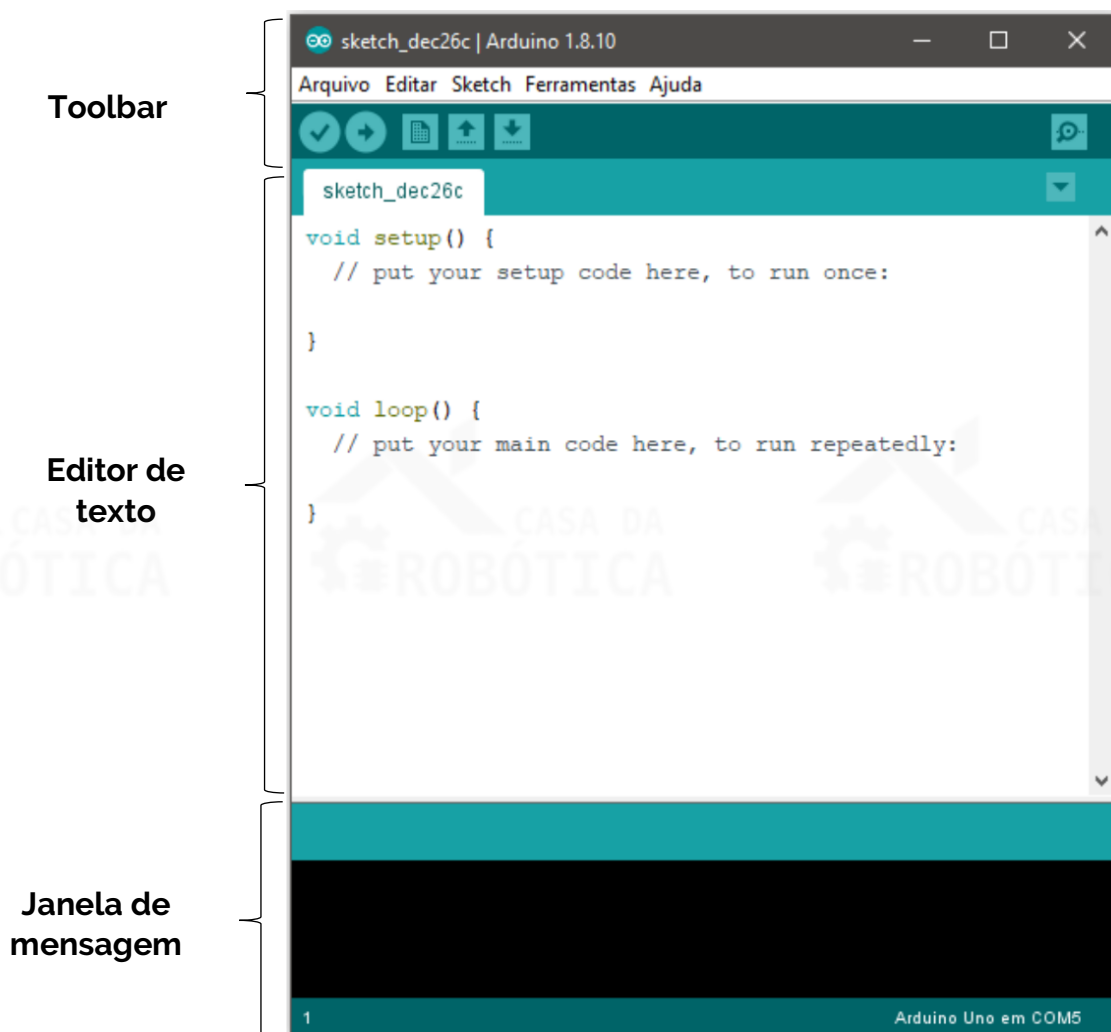
Em nosso blog, você encontrará o passo a passo para instalar o Arduino IDE no computador Windows. Acesse:

www.blogdarobotica.com/instalando-o-arduino-ide

EXPLORANDO O ARDUINO IDE

Ao abrir o Arduino IDE, você verá uma tela semelhante à Figura 6. Caso esteja utilizando o Linux ou Mac OS X, pode haver pequenas diferenças, mas o IDE é basicamente o mesmo para todos os sistemas operacionais.

Figura 6 - Visual do Arduino IDE.



O Arduino IDE pode ser dividido em três partes: A Toolbar no início da tela, o editor de texto no centro e a janela de mensagens na base.

No top da Toolbar há uma barra de menus contendo comandos comuns com os itens: Arquivo, Editar, Sketch, Ferramentas e Ajuda. Os comandos e funções disponíveis na barra de ferramenta podem ser consultados ao acessar o comando Ajuda > Ambiente.

Ainda na Toolbar encontra-se os botões de atalho, que fornecem acesso rápido às funções mais utilizadas. A seguir são mostrados os ícones e o detalhamento de suas funções.



Verificar: Analisa se há erros em seu código



Upload: Compila seu código e o envia para a placa microcontroladora



Novo: Cria um novo Sketch



Abrir: Mostra uma lista de Sketch existentes



Salvar: Salva o seu Sketch atual



Monitor serial: Exibe os dados seriais enviados pela placa microcontroladora

O editor de texto é o campo destinado a escrita dos códigos. Os códigos escritos usando Arduino ou placas compatíveis são conhecidos como Sketches e são salvos com a extensão de arquivo .ino. Este editor de texto tem características de um editor tradicional, contendo funções de cortar, copiar, colar, selecionar tudo, entre outras.

A janela de mensagem fornece mensagens de feedback ao salvar e exportar arquivos, bem como exibe informações de erros no código ou ao compilar.

Conectando a placa UNO ao computador

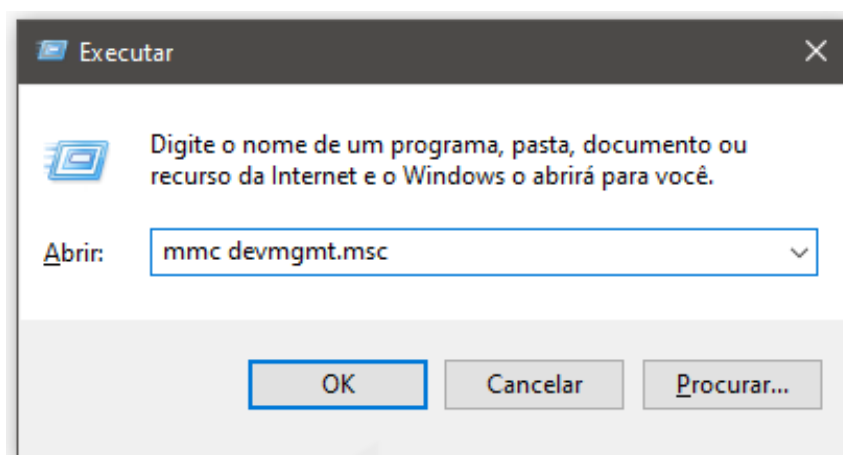
Agora que já conhecemos a placa UNO e sua interface de desenvolvimento, vamos conectá-lo ao computador. Esta conexão é realizada por meio de um cabo USB do tipo A para o tipo B, igual ao da Figura 7.

Figura 7 - Cabo USB do tipo A para o tipo B.



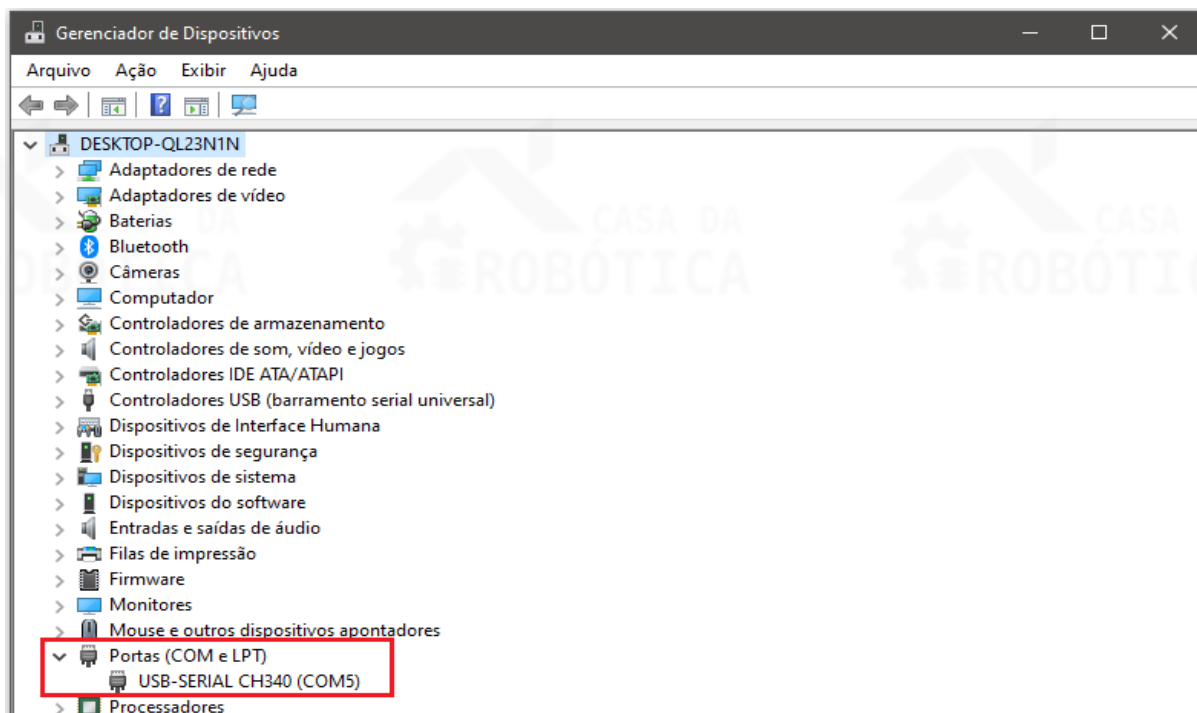
Conecte sua placa UNO ao seu computador através da USB. Para saber se o seu computador Windows está identificando a placa vamos realizar um teste acessando o gerenciador de dispositivos. Uma opção para se chegar neste painel é pressionar as teclas “Windows + r”. Assim que o menu executar abrir digite “mmc devmgmt.msc” sem as aspas, como se pode ser observado na Figura 8.

Figura 8 - Atalho para acessar o gerenciador de dispositivos.



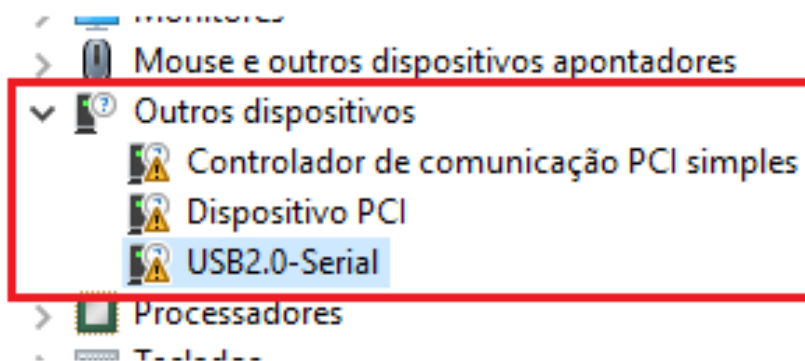
Após digitar esse comando e clicar em “OK” será aberta a tela da Figura 9. Para verificar se o driver da placa UNO foi reconhecido navegue até a opção Portas (COM e LPT) e expanda clicando na setinha do lado do nome. No exemplo abaixo a placa UNO foi reconhecida com sucesso pela porta COM de número 5, essa informação será útil posteriormente.

Figura 9 - Tela do gerenciador de dispositivos.



Caso a placa UNO não seja reconhecida pelo seu computador, ela pode aparecer, com o  ícone, em "Outros dispositivos", como na Figura 10.

Figura 10 - Indicador de que a placa Arduino não foi reconhecida.



Isso acontece devido à falta de um driver para a interpretação do dispositivo. Para resolver esse problema basta baixar o driver disponível no link a seguir e executá-lo:

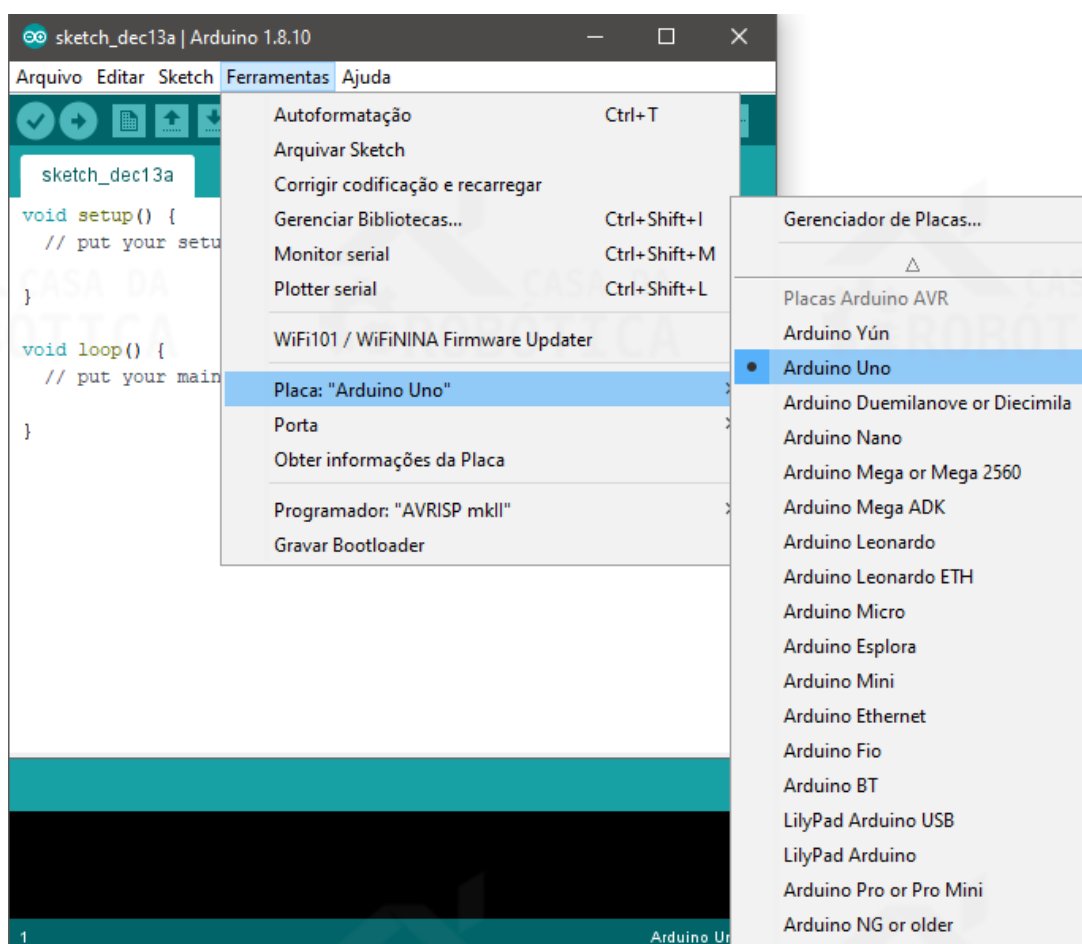
www.blogdarobotica.com/instalando-o-driver-serial-para-arduino/

Após a conclusão do download, execute os arquivos baixados, instale os drivers e reinicie o computador.

Seleção da placa e da porta de comunicação da placa UNO

Agora que já temos a placa UNO conectada ao computador, vamos selecionar a placa e a porta de comunicação no Arduino IDE. Para tal, deve-se selecionar o modelo da placa utilizada no menu Ferramentas, para nossos exemplos usaremos o Arduino Uno, conforme Figura 11.

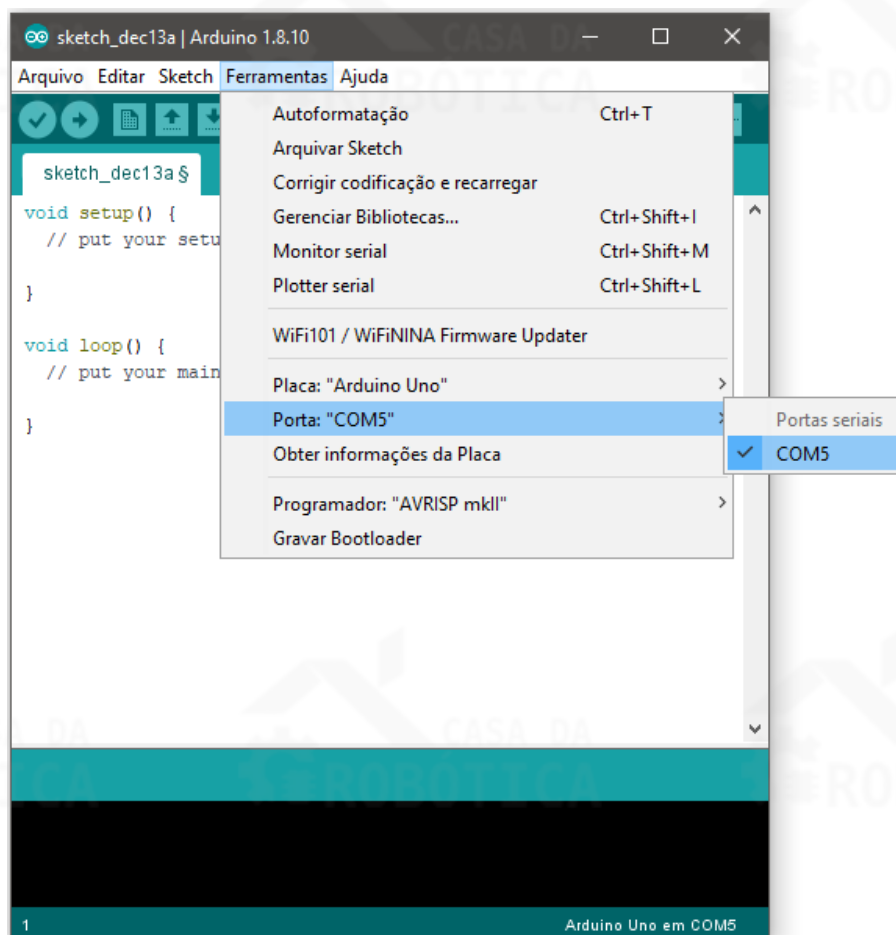
Figura 11 - Seleção da placa Arduino.



Após a seleção do modelo, deve-se selecionar a porta de comunicação a placa foi atribuída, ou seja, a porta que a placa UNO foi reconhecida. Como vimos

anteriormente, em nosso exemplo, a placa UNO foi reconhecida pela COM de número 5. A Figura 12 mostra a seleção da COM através do menu Ferramentas.

Figura 12 - Seleção da porta COM.



Após isto, sua placa UNO estará pronta e o ambiente de desenvolvimento configurado para uso.

CONHECENDO OS DEMAIS COMPONENTES

O QUE É O MOTOR DC?

Um motor DC, ou motor de corrente contínua, nada mais é do que um equipamento capaz de converter energia elétrica de corrente contínua em energia mecânica.

Pequenos motores DC são amplamente utilizados em ferramentas, brinquedos, eletrodomésticos, em projetos com Arduino e robótica. O motor DC 3 a 6V com caixa de redução (Figura 13) é ideal para construção de seguidores de linha, carrinhos de controle remoto, plataformas robóticas, esteiras, entre outros.

Figura 13 - Motor DC 3 a 6V com caixa de redução.



As principais características do motor DC 3 a 6V são descritas a seguir:

- Tensão de alimentação: 3V a 6V;
- Corrente em aberto: 190mA;
- Corrente máxima com carga: 350mA;
- Relação da caixa de redução: 48:1;
- Velocidade: 140RPM a 4,5V (sem carga);
- Torque: 800gf.cm;

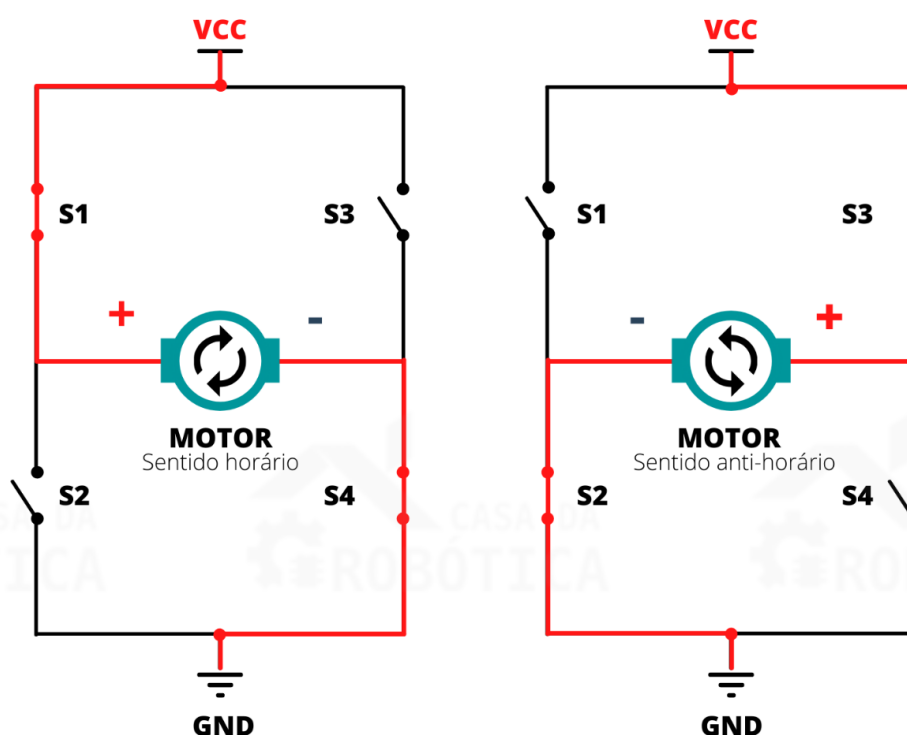
Para ter um controle completo de um motor DC, temos que controlar seu sentido de rotação e velocidade. Para tal, combinamos duas técnicas:

- Ponte H – Para controlar o sentido de rotação;
- PWM – Para controlar a velocidade.

O motor DC 3 a 6 V com caixa de rotação pode rotacionar tanto no sentido horário quanto no sentido anti-horário. O sentido da rotação desse motor é determinado pelo sentido da corrente elétrica que circula em seus enrolamentos. Deste modo, para alterar o sentido de rotação do motor é necessário mudar o sentido da corrente elétrica que o alimenta. Em outras palavras, se invertermos a polaridade de alimentação do motor DC, seu sentido de rotação também será invertido.

Usar uma ponte H é uma técnica comum para alterar a polaridade da tensão de entrada do motor DC. O circuito de uma ponte H (Figura 14) contém quatro chaves e por meio da comutação destas conseguimos alterar a rotação do motor. Conforme pode ser observado no circuito abaixo, se queremos que o motor gire no sentido horário precisamos fechar as chaves S1 e S4. No entanto, para rotacionar o motor no sentido anti-horário, fechamos as chaves S3 e S4.

Figura 14 - Circuito elétrico de um circuito ponte H.

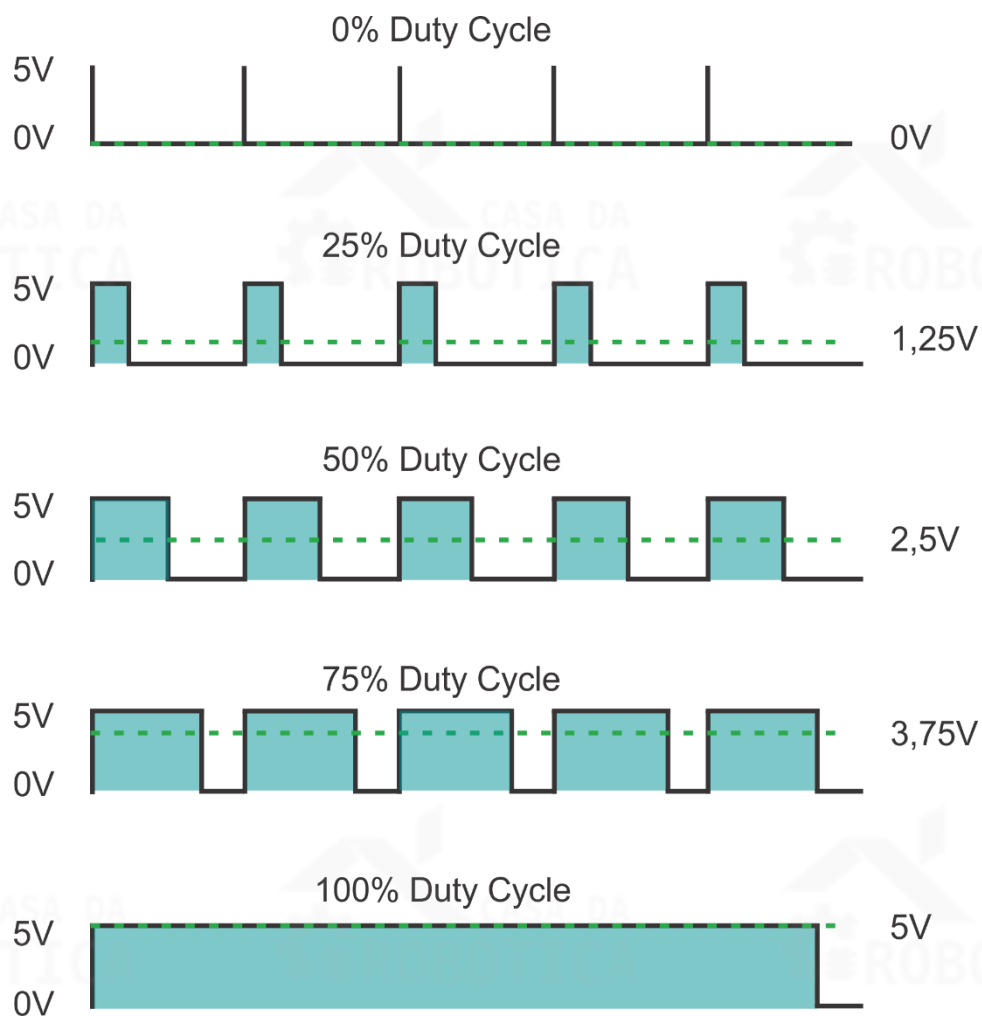


A velocidade do motor DC pode ser controlada variando sua tensão de entrada e uma técnica comum para fazer isso é usar PWM (Modulação por largura de pulso). O PWM é uma técnica em que o valor médio da tensão (e corrente) de entrada do motor é controlado enviando uma série de pulsos ligados e desligados (ON-OFF).

A tensão média proporcionar à largura dos pulsos é conhecida como Duty Cycle (ciclo de trabalho). Quanto maior é o Duty Cycle, maior é a tensão média aplicada ao motor DC e maior será a sua velocidade de rotação. De forma contrária, quando menor o Duty Cycle, menor é a tensão média aplicada ao motor DC e menor sua velocidade de rotação.

A Figura 15 ilustra a técnica PWM com vários Duty Cycle e tensões médias.

Figura 15 - Técnica PWM com vários ciclos de trabalho e tensões médias.



O QUE É O SHIELD PONTE H L298P?

A Ponte H é considerada um dos circuitos mais importantes na elaboração de um sistema automatizado, utilizada para controle e acionamento de um motor de corrente contínua a partir de um sinal gerado pelo microcontrolador. Conforme vimos anteriormente, a seleção do sentido de rotação de um motor DC é facilmente obtida invertendo-se a polaridade sobre terminais do circuito.

A ponte H é um circuito eletrônico de potência utilizado para controle e acionamento de motor DC a partir de um sinal gerado pelo microcontrolador. Seu uso é necessário porque os sinais de saída dos microcontroladores não suportam a corrente exigida para acionamento da maioria dos motores. Portanto, a ponte H funciona como uma unidade de potência que pode alimentá-lo convenientemente.

Atualmente, existem muitos modelos de módulos e Shields ponte H para Arduino, como o Shield Ponte H L298P (Figura 16).

Figura 16 - Shield Ponte H L298P com Buzzer para Controle de Motor DC.



O Shield Ponte H L298P foi desenvolvido para encaixar diretamente nas placas UNO ou outro Arduino com pinos de entrada e saída compatíveis. Este

e parada de dois motores DC de forma independente. Uma grande vantagem na utilização deste Shield é que ele possui bornes para alimentação externa, uma vez que as placas compatíveis com o Arduino não suportam a corrente exigida pelos

motor, ultrassônico, LED RGB, sensores analógicos/digitais e Bluetooth. A Figura 17 indica os pinos do Shield Ponte H L298P, que serão detalhados em seguida.



Ponte H L298P, dos motores e da placa UNO conectada. A tensão máxima de



OBSERVAÇÃO:

Respeitar a faixa de tensão especificada garante o correto funcionamento do Shield e evita o superaquecimento do regulador da placa UNO.

GND

O GND do borne de entrada representa o terra do Shield. Os demais pinos GND fornecem potencial de terra aos dispositivos externos.

SER

O SER corresponde aos conectores disponíveis para uso de um servo motor, motor utilizado em várias aplicações quando se deseja movimentar algo de forma precisa e controlada.

O acionamento de um servo motor exige um pino de alimentação de 5V, um de aterramento e um de controle PWM. O Shield usa o pino 9 da placa UNO para o sinal PWM e possui um regulador de tensão de 5V dedicado para alimentar o servo, o que evita que ruídos elétricos voltem para a alimentação lógica principal de 5V.

Os pinos SER são:

- **G:** Terra;
- **+5:** 5V dedicado à alimentação do servo motor;
- **9:** Pino digital 9 da placa UNO.

Caso você não necessite utilizar um servo motor em seu projeto, os pinos SER estarão disponíveis para outros usos.

ENA e ENB

O sinal de controle de direção dos motores é realizado através dos pinos ENA e ENB do Shield, que se encontram conectados respectivamente aos pinos 12 e 13 da placa UNO.

O ENA (pino 12) é responsável pela direção do motor A. Quando colocado em nível alto (HIGH), o motor girará em um sentido. Para girar no sentido inverso, basta definir nível baixo (LOW) no pino.

O ENB (pino 13) é responsável pela direção do motor B. O motor B girará em um sentido quando colocado em nível alto (HIGH) e inverterá sua rotação quando colocado em nível baixo (LOW).

PWMA e PWMB

Como vimos anteriormente, a velocidade de um motor DC pode ser controlada variando sua tensão de entrada através da técnica PWM (Modulação por largura de pulso). O Shield Ponte H possui dois pinos PWM dedicados ao controle de velocidade dos motores.

O PWMA está conectado ao pino digital 10 da placa UNO e controla a velocidade do motor A.

O PWMB está conectado ao pino digital 11 da placa UNO e controla a velocidade do motor B.

ULT

O ULT representa os conectores disponíveis para uso de um ultrassônico, que utiliza alta frequência do som para realizar a detecção de objetos e sua distância. O Shield possui 4 pinos dedicados a conexão de um ultrassônico do tipo HC-SR04, são eles:

- **V**: Alimentação 5V;
- **R**: pino Echo - leitura do tempo de resposta (Pino digital 8);
- **T**: Pino Trig - gatilho de disparo do pulso (Pino digital 7);
- **G**: Terra.

RBG

O Shield Ponte H L298P também possui conectores para uso de um LED RGB. Os LEDs RGB consistem na junção de três LEDs em um só dispositivo, mas que podem ser controlados individualmente. Cada um destes LEDs possuem uma cor distinta: Um vermelho (*Red*), um verde (*Green*) e um azul (*Blue*).

O Shield possui 5 pinos para acionamento de LEDs RGB do tipo cátodo comum ou ânodo comum:

- **V**: Alimentação 5V para LED RGB do tipo ânodo comum. Este pino não possui nomenclatura, mas está localizado ao lado do pino digital 7;
- **B**: Acionamento do LED interno azul (pino digital 6);

- **G:** Acionamento do LED interno verde (pino digital 5);
- **-:** Terra para LED RGB do tipo cátodo comum;
- **R:** Acionamento do LED interno vermelho (pino digital 3).

BUZ

O Shield Ponte H possui um buzzer ativo integrado, representado na Figura 17 pela sigla BUZ. Este buzzer pode ser ativado colocando a porta digital 4 da placa UNO em nível alto (HIGH).

BT2

O BT2 é um conector fêmea disponível para utilização de um módulo Bluetooth de alimentação de 3,3V. Os pinos do BT2 incluem:

- **+**: Alimentação 3,3V;
- **-:** Terra;
- **T:** Tx – Envio de dados (pino digital 0);
- **R:** Rx – Recebimento de dados (pino digital 1).



OBSERVAÇÃO:

O módulo Bluetooth usado no BT2 apenas funcionará corretamente se a sua alimentação for de 3,3V. Caso seu módulo Bluetooth necessite de uma alimentação de 3,6 a 6V não plugue diretamente no BT2 e o alimente usando o pino 5v da placa UNO.

D2

O conector representados por D2 possui 3 pinos (5V, GND e pino digital 2) que pode ser usado para um sensor digital ou outra conexão remota.

- **G:** Terra;
- **+**: Alimentação 5V;
- **S:** Pino digital 2 da placa UNO.

GND, 5V e A/D

Outra grande vantagem do Shield Ponte H L298P é que ele possui varios pinos terra e alimentação 5V. Esta placa traz também os pinos A0-A5, que podem ser usados para entradas analógicas ou como entrada/saída digital.

GND, 5V e 3.3V

O Shield Ponte H L298P possui pinos GND, 5V e 3.3V seguindo a pinagem da placa UNO.

RESET

O RESET é um botão de fácil acesso para reiniciar a placa UNO.

MOTORA e MOTORB: MA1, MA2, MB1 e MB2

Os bornes identificados por MOTORA e MOTORB devem ser utilizados para a conexão do Shield com os motores. Os pinos positivos e negativos para cada motor não são rotulados, você pode conectar da maneira que preferir. Se o motor for em direção oposta a que você espera, basta inverter os fios do motor.

- MA1 e MA2: Conexão do motor A;
- MB1 e MB2: Conexão do motor B.

O QUE O MÓDULO SEGUIDOR DE LINHA COM SENSOR INFRAVERMELHO TCRT5000?

Um sensor infravermelho é um dispositivo eletrônico que emite e/ou detecta radiação infravermelha com o intuito de revelar alguns aspectos do ambiente a seu redor. Esses sensores podem ser utilizados para detectar movimentos, medir o calor de um objeto, ler de código de barras, em alarmes de passagem, entre outros.

No campo da robótica, o sensor infravermelho é amplamente utilizado na construção de robôs seguidores de linha. Os seguidores de linha são robôs pré-programados para detectar e seguir uma linha, geralmente branca ou preta. A identificação da cor da linha pode ser feita usando sensores infravermelho, como o TCRT5000 (Figura 18).

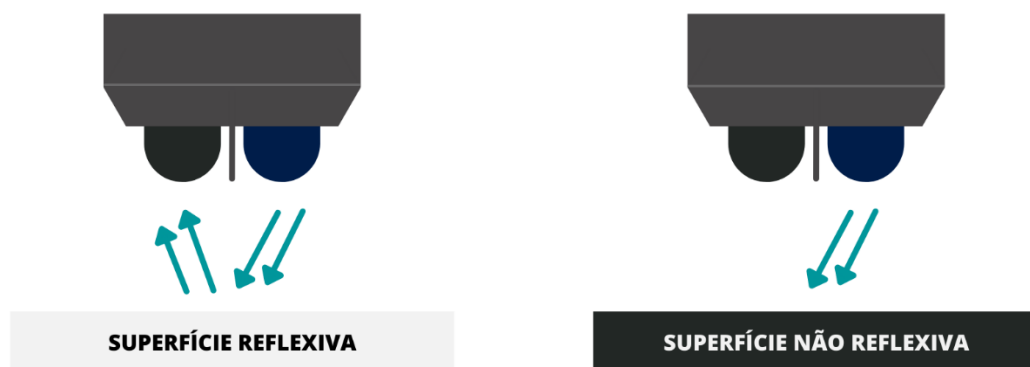
Figura 18 – Sensor infravermelho TCRT5000.



O sensor infravermelho TCRT5000 é formado por dois componentes: um LED emissor infravermelho (de cor azul), que emite um feixe de luz numa frequência não visível a olho nu, e um receptor fototransistor (de cor preta), que captura o feixe de luz emitido pelo infravermelho.

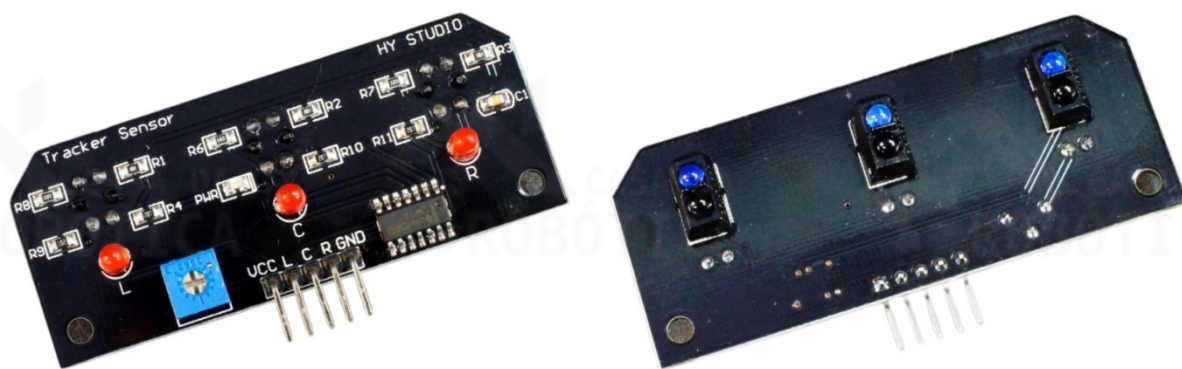
Estes dois componentes funcionam em conjunto. O LED emite a luz infravermelha que é refletida por um objeto, posicionado em frente ao sensor, que é detectada pelo fototransistor. A cor e o material do objeto podem interferir no funcionamento do TCRT 5000. Por exemplo, os objetos de cor branca são bons refletores, ao contrário dos objetos de cor preta.

Figura 19 - Funcionamento do sensor TCRT5000 em diferentes superfícies.



O módulo seguidor de linha infravermelho (Figura 20) é uma placa de suporte para 3 sensores TCRT5000. Este componente foi projetado para uso com robôs de seguidores de linha, permitindo que eles sigam opticamente uma linha no chão. Seguir linha é uma das maneiras mais simples de implementar a navegação de um robô.

Figura 20 - Módulo seguidor de linha com sensor infravermelho TCRT5000.



O módulo seguidor de linha infravermelho possui bom alcance e sensibilidade. Seus sensores possuem alcance de até 4 cm (em linhas pretas e brancas). Este componente possui um potenciômetro integrado para ajustar a sensibilidade dos sensores, 3 LEDs indicadores, um CI LM393 e cinco pinos de conexão, indicadas por:

- **VCC:** Tensão de alimentação 5V;
- **L:** Saída digital do sensor infravermelho esquerdo;
- **C:** Saída digital do sensor infravermelho central;
- **R:** Saída digital do sensor infravermelho direito;
- **GND:** Terra.

O princípio de funcionamento do módulo seguidor de linha infravermelho é o seguinte: Os LEDs emissores emitem a luz infravermelha continuamente, quando essa luz não for refletida ou não for forte o suficiente, o sensor terá sua saída digital em nível alto (HIGH) e o LED correspondente desligará, indicando que a linha foi detectada.

O QUE É O SUPORTE PARA 6 PILHAS AA?

O suporte para 6 pilhas AA (Figura 21) fornece alimentação para os circuitos eletrônicos, sendo amplamente utilizado em projetos de robótica, automação residencial, internet das coisas, entre outros.

O suporte para 6 pilhas AA possui rabichos (fios negativo e positivo), chave liga/desliga e tampa de proteção das pilhas. Neste suporte as 6 pilhas são ligadas em série.

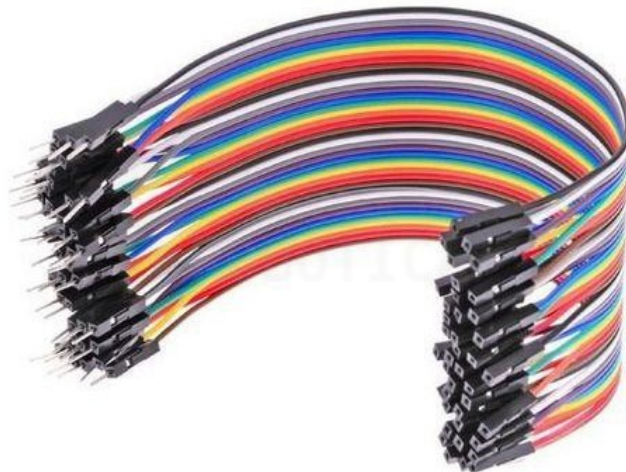
Figura 21 - Suporte para 6 pilhas AA.



O QUE SÃO OS JUMPERS?

Os jumpers (Figura 22) são pequenos fios condutores utilizados para conectar dois pontos de um circuito eletrônico. Os jumpers facilitam a conexão entre componentes elétricos, sendo uma excelente escolha para montagem de projetos e interligação do Módulo Seguidor de Linha com o Shield Ponte H L298P.

Figura 22 - Jumpers.



ESQUEMÁTICO ELÉTRICO DO CDR CAR

Agora que já conhecemos todos os componentes eletrônicos que farão o CDR CAR funcionar, podemos realizar a montagem eletrônica. Inicie montagem do circuito da Figura 23 com a **placa UNO desconectada e sem as pilhas no suporte**. A montagem ou modificação das ligações elétricas com o circuito energizado pode ocasionar danos aos componentes do CDR CAR.

Figura 23 - Esquemático elétrico do CDR CAR.

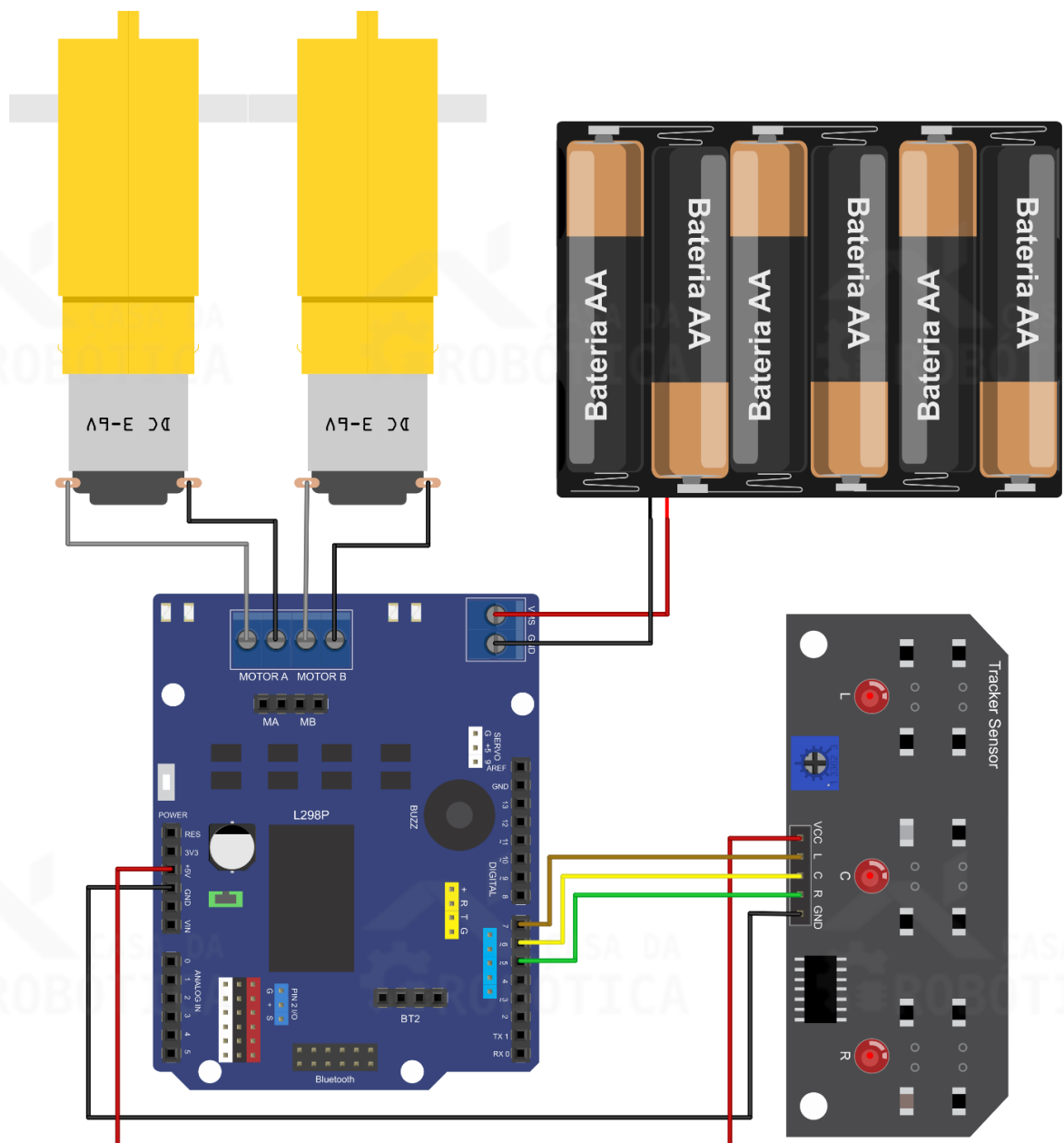


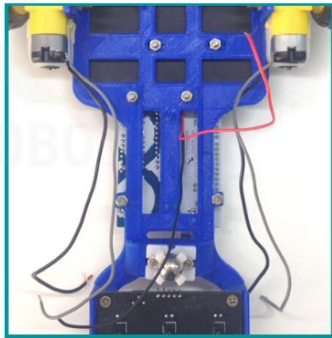
Tabela 1 - Conexões elétricas do CDR CAR.

	Pino	Shield Ponte H
Motores	Esquerdo	MOTORA
	Direito	MOTORB
Módulo seguidor de linha	VCC	5 V
	L	Pino digital 5
	C	Pino digital 6
	R	Pino digital 7
	GND	GND
Suporte para pilhas	VCC	VMS
	GND	GND

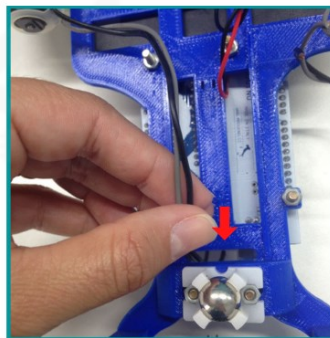
Ao montar seu circuito do CDR CAR preste **atenção** nos seguintes pontos:

- Você pode utilizar fios de cores diferentes, mas deve assegurar que os componentes e fios estejam conectados como na Figura 23;
- Os fios do motor esquerdo serão conectados nas entradas do borne do Shield Ponte H identificado por MOTORA. A ordem e cores dos fios do motor esquerdo não importam neste momento, contanto que eles sejam conectados às entradas do borne MOTORA;
- Os fios do motor direito deverão ser conectados nas entradas do borne MOTORB. A ordem e cores dos fios do motor direito não importam neste momento, contanto que eles sejam conectados às entradas do borne MOTORB;
- O Módulo Seguidor de Linha deve ser alimentado por uma tensão de 5V. Portanto, os pinos VCC e GND deste componente devem ser conectados ao pino 5V e GND do Shield Ponte H, respectivamente. As saídas dos sensores infravermelhos do módulo representadas por L (esquerdo), C (centro), R (direito) deverão ser conectadas aos pinos digitais 5, 6 e 7 do Shield Ponte H, nesta ordem;
- O suporte para 6 pilhas AA fornecerá a alimentação para o circuito eletrônico do CDR CAR. Este componente possui dois fios, um vermelho e um preto. O fio vermelho corresponde a tensão positiva (VCC) e deve ser conectado ao pino VMS do Shield Ponte H. O fio preto representa o GND e deve ser conectado ao pino GND do Shield Ponte H.

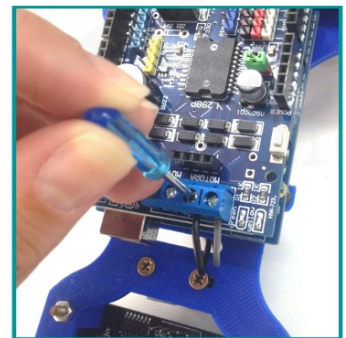
PASSO A PASSO PARA MONTAGEM ELETRÔNICA DO CDR CAR



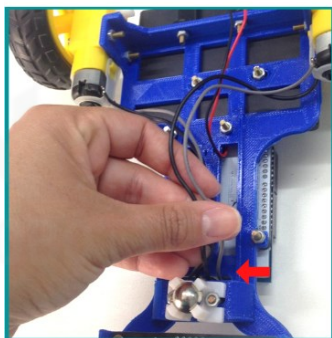
- 1** Após a montagem mecânica, os fios dos motores e do suporte de pilha deverão ficar na parte inferior do CDR CAR



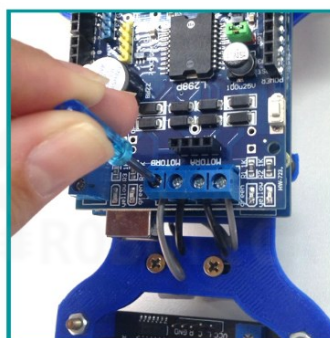
- 2** Passe os fios do motor esquerdo através da abertura localizada acima da roda boba



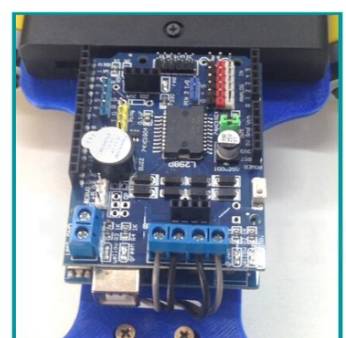
- 3** Conecte os fios do motor esquerdo nas entradas do borne **MOTORA** do Shield Ponte H



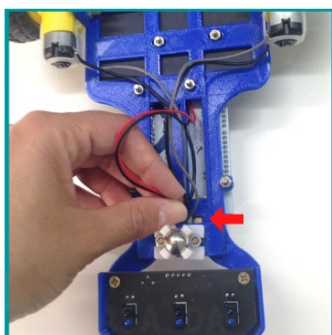
- 4** De mesmo modo, passe os fios do motor direito através da abertura localizada acima da roda boba



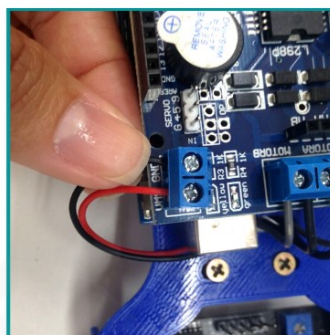
- 5** Conecte os fios do motor direito nas entradas do borne **MOTORB** do Shield Ponte H



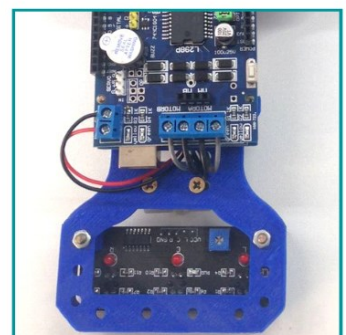
- 6** Os fios dos motores devem ficar como na imagem acima.



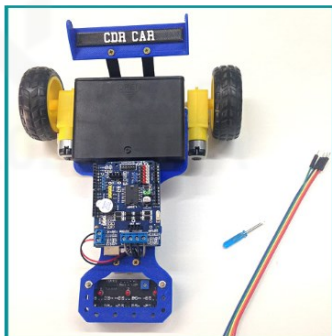
- 7** Em seguida, passe também os fios do suporte de pilhas através da abertura localizada acima da roda boba



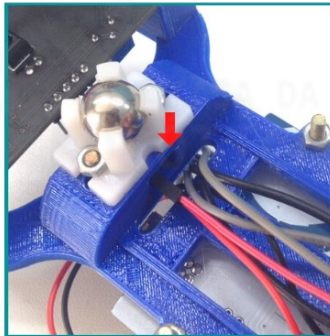
- 8** Conecte o fio vermelho à entrada **VMS** e o fio preto à entrada **GND** do Shield Ponte H



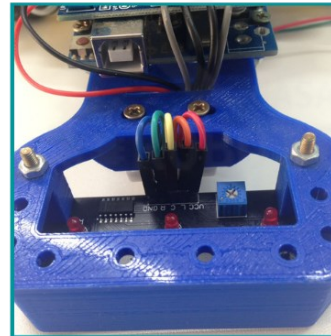
- 9** Os fios do suporte de pilhas devem ficar como na imagem acima.



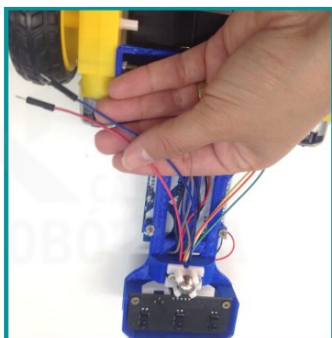
- 10** Separe 5 jumpers macho x fêmea para conexão do Módulo Seguidor de Linha



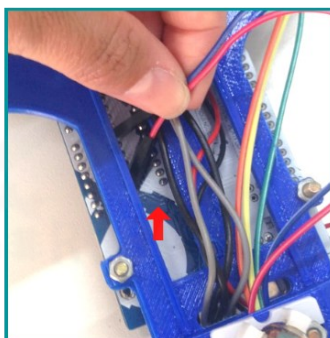
- 11** Passe um jumper por vez através da abertura localizada abaixo da roda boba



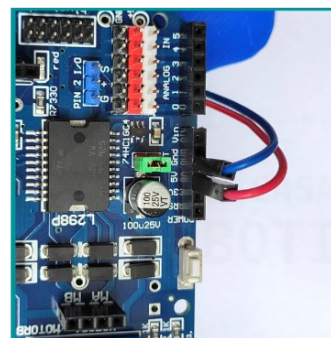
- 12** Conecte os jumpers aos pinos do Módulo Seguidor de Linha



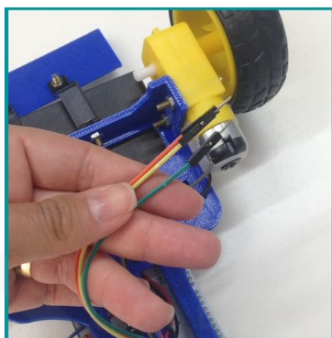
- 13** Separe os jumpers conectados aos pinos **VCC** e **GND** do Módulo Seguidor de Linha



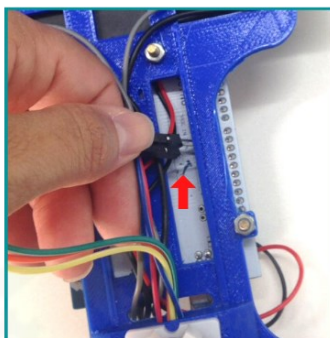
- 14** Passe esses jumpes através da abertura esquerda no centro do chassi do CDR CAR



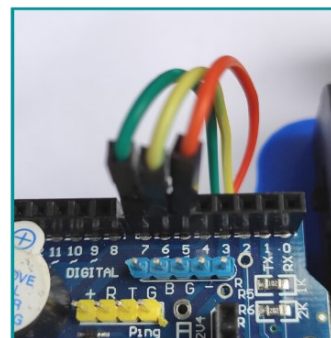
- 15** Conecte o jumper **VCC** ao pino **5V** e o jumper **GND** ao pino **GND** do Shield



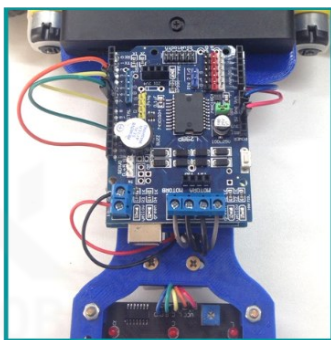
- 16** Separe os jumpers conectados aos pinos **L**, **C** e **R** do Módulo Seguidor de Linha



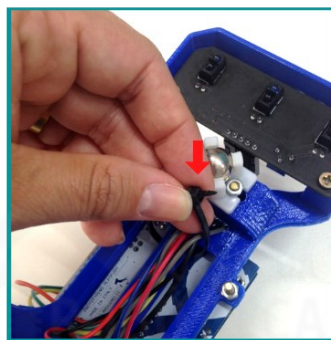
- 17** Passe esses jumpes através da abertura direita no centro do chassi do CDR CAR



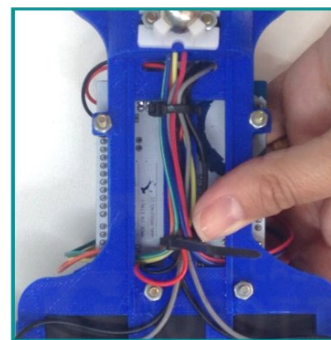
- 18** Conecte o jumper **L**, **C** e **R** aos pinos digitais **5**, **6** e **7** do Shield Ponte H, nesta ordem



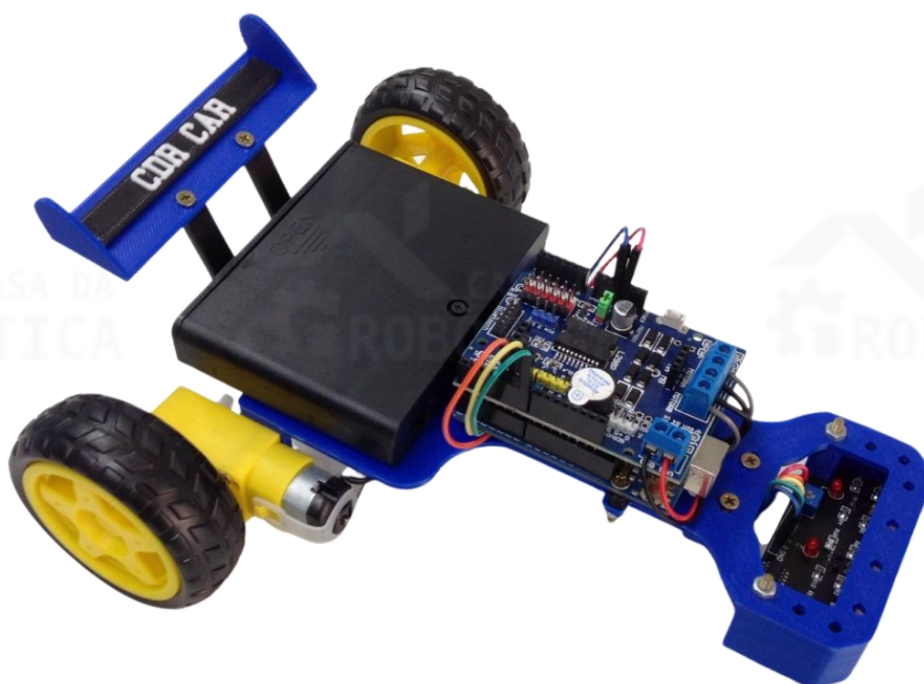
19 Os jumpers do Módulo Seguidor de Linha devem ficar como na figura acima



20 Para organizar os fios, puxe-os levemente para a parte inferior do CDR CAR e prenda-os com as abraçadeiras



21 As abraçadeiras devem ficar como na figura acima



ATENÇÃO:

Apenas conecte as pilhas e ligue o CDR CAR quando tiver certeza de que as ligações estão corretas.

4. FUNDAMENTOS DE PROGRAMAÇÃO

PROGRAMAÇÃO PARA ARDUINO

Programação pode ser definida como o processo de projetar, escrever, verificar e manter um código ou programa que comande as ações de uma máquina, como computador, celular, microcontrolador, entre outras.

Um código, programa ou sketch – como são denominados os códigos em Arduino, são compostos por um conjunto de instruções sequenciais definido pelo programador que descrevem as tarefas que a máquina deve realizar. Em outras palavras, para que a máquina execute comandos específicos é necessário elaborar um programa escrito contendo todas as instruções, ordenadas sequencialmente, em uma linguagem de programação.

Uma linguagem de programação é um idioma artificial desenvolvido para prover a comunicação entre uma pessoa (programador) e uma máquina (computador, microcontrolador, etc.). Atualmente, existem centenas de linguagens de programação e em todas elas o programador pode definir como a máquina deve atuar, como armazenar ou transmitir os dados, quais ações tomar em diferentes situações, quando finalizar a sua operação, entre outras. Dentre as linguagens de programação mais populares pode-se citar o C, C++, C#, Java, JavaScript, Python, PHP.

Os programas para microcontroladores compatíveis com o projeto Arduino, como o microcontrolador UNO R3 Atmega328 incluso neste Kit Iniciante, são implementados tendo como referência a linguagem de programação C++, mantendo preservada sua sintaxe clássica de declaração de variáveis, nos operadores, nas estruturas e em muitas outras características.

Antes de aprendermos a linguagem de programação do Arduino devemos conhecer alguns elementos básicos que compõem um Sketch. Desta forma, a seguir apresentaremos a estrutura de um sketch, variáveis, funções e bibliotecas.

ESTRUTURAS BÁSICA DE UM SKETCH

Todos os Sketch Arduino devem ter a estrutura composta pelas funções `setup()` e `loop()`, conforme ilustra o exemplo a seguir.

```
1  void setup()
2  {
3      Comando 1;
4      Comando 2;
5      ...
6  }
7  void loop()
8  {
9      Comando 3;
10     Comando 4;
11     ...
12 }
```

A função `setup()` é executada apenas uma vez na inicialização do programa, e é nela que você deverá descrever as configurações e instruções gerais para preparar o programa antes que o loop principal seja executado. Em outras palavras, a função `setup()` é responsável pelas configurações iniciais da placa microcontroladora, tais como definições de pinos de entrada e saída, inicialização da comunicação serial, entre outras.

A função `loop()` é a função principal do programa e é executada continuamente enquanto a placa microcontroladora estiver ligada. É nesta função que todos os comandos e operações deverão ser escritos.



OBSERVAÇÕES:

- As instruções dadas em um Sketch são realizadas de forma sequencial;
- É necessário incluir o sinal ; (ponto e vírgula) para que o programa identifique onde uma instrução deve ser finalizada.

VARIÁVEIS

As variáveis são expressões que podemos utilizar em programas para nomear e armazenar um dado para uso posterior, como dados de um sensor ou um valor calculado.

Antes de serem utilizadas, todas as variáveis devem ser declaradas, definindo seu tipo e, opcionalmente, definindo seu valor inicial. Alguns tipos de variáveis encontram-se listadas na Tabela 2.

Tabela 2 - Tipos de variáveis e suas descrições.

Tipo	Descrição
<code>char</code>	Utilizado para armazenar um valor de caractere.
<code>byte</code>	Usado para armazenar um número entre 0 e 255.
<code>int</code>	Utilizado para armazenar números inteiros.
<code>bool</code>	Empregado para armazenar dois valores: <code>true</code> (verdadeiro) ou <code>false</code> (falso).
<code>float</code>	Armazena números decimais que ocupam 32 bits (4 bytes).
<code>double</code>	Armazena números decimais que ocupam 64 bits (8 bytes).
<code>void</code>	Usada apenas em declarações de funções
<code>String</code>	Utilizada para armazenar cadeias de texto.

A declaração de uma variável pode ser melhor entendida com o exemplo a seguir. Neste caso, declaramos uma variável com nome `a` do tipo `int`, uma variável com nome `b` do tipo `float` e uma variável de nome `c` do tipo `char`. Quando as variáveis são declaradas antes da função `setup()`, significa que elas são variáveis globais, que podem ser usadas em todas ao longo de todo o programa. Por sua vez, as variáveis declaradas em funções específicas, como no `loop()`, são variáveis locais e só podem ser usadas dentro da sua função de origem.

```
1  int a;  
2  float b;  
3  char C;  
4  
5  void setup()  
6  {  
7      ...  
8  }
```

Para definir um valor inicial a variável utilizamos o comando de atribuição, representado pelo símbolo `=`. Desta forma, quando escrevemos `int a = 10;` estamos atribuindo o valor 10 a variável de nome `a` do tipo `int`.



OBSERVAÇÕES:

- O nome das variáveis deve seguir algumas regras: Devem iniciar com letras ou sublinhado `_` e não podem ter nome idêntico a alguma das palavras reservadas pelo programa ou biblioteca;
- Letras maiúsculas e minúsculas fazem diferença. Se o nome da variável definida for algo como, por exemplo, `ledPin`, não será a mesma coisa que `LedPin` ou `LEDPIN`;
- O símbolo `=` tem o papel exclusivo de atribuição. A igualdade matemática é representada pela dupla igualdade `==`.

FUNÇÕES

Funções são blocos de instruções que podem ser chamados em qualquer parte do seu Sketch. A principal motivação para o uso de funções em um programa é quando necessitamos executar a mesma ação várias vezes. Desta forma, a segmentação do código em funções permite a criação de trechos de código modulares que executam uma tarefa definida e retornam à área do código a partir da qual a função foi chamada.

O uso de funções possui várias vantagens, entre elas:

- As funções ajudam o programador a manter o sketch organizado;
- As funções codificam uma ação em um só lugar, de forma que o trecho do código precise ser pensado e escrito apenas uma vez;
- Reduz as chances de erros na modificação quando o código precisa ser alterado;
- Facilitam a reutilização de código em outros programas;
- Tornam o código mais legível.

As duas funções principais na criação de um sketch no Arduino são `void setup()` e `void loop()`, mas existem algumas outras funções predefinidas para controlar uma placa microcontroladora, conforme mostra a Tabela 3:

Tabela 3 – Funções e suas descrições.

Controle	Função	Descrição
Entrada e saída digitais	<code>digitalRead()</code>	Lê o valor de um pino digital especificado
	<code>digitalWrite()</code>	Escreve no pino digital <code>HIGH</code> ou <code>LOW</code>
	<code>pinMode()</code>	Configura o pino para funcionar como saída ou entrada
Entrada e saída analógicas	<code>analogRead()</code>	Lê o valor de um pino analógico especificado
	<code>analogReference()</code>	Configura a tensão de referência para a entrada analógica
	<code>analogWrite()</code>	Aciona uma onda PWM em um pino
Funções temporizadoras	<code>delay()</code>	Pausa o programa por um período (em milissegundos)
	<code>millis()</code>	Retorna o número de milissegundos passados desde que a placa Arduino começou a executar o programa.
Entradas e saídas avançadas	<code>tone()</code>	Gera uma onda quadrada na frequência especificada em um pino
	<code>noTone()</code>	Interrompe a função <code>tone()</code>
Funções matemáticas	<code>map()</code>	Mapeia um intervalo numérico em outro intervalo desejado
	<code>sq()</code>	Calcula o quadrado de um número
	<code>sqrt()</code>	Calcula a raiz quadrada de um número
Funções trigonométricas	<code>cos()</code>	Calcula o cosseno de um ângulo (em radianos)
	<code>sin()</code>	Calcula o seno de um ângulo (em radianos)
	<code>tan()</code>	Calcula a tangente de um ângulo (em radianos)
Números aleatórios	<code>random()</code>	Gera números pseudoaleatórios.
	<code>randomSeed()</code>	Inicializa o gerador de números pseudoaleatórios
Comunicação	<code>Serial</code>	Usada para comunicação entre uma placa Arduino e um computador ou outros dispositivos
Interrupções Externas	<code>attachInterrupt()</code>	Cria interrupção externa
	<code>detachInterrupt()</code>	Desativa a interrupção externa
Interrupções	<code>interrupts()</code>	Reativa interrupções
	<code>noInterrupts()</code>	Desativa interrupções



OBSERVAÇÕES:

- Para saber mais sobre as funções predefinidas do Arduino acesse o site: <https://www.arduino.cc/reference/pt/>. Nele você encontrará outras funções não especificadas neste material de apoio e poderá consultar a descrição, sintaxe e parâmetros das funções que desejar;

- Os sinais analógicos são aqueles que variam continuamente ao longo do tempo. Por sua vez, os sinais digitais assumem valores discretos (0 ou 1);
- Configurar um pino digital em **HIGH** significa colocar o pino digital em nível lógico alto (1), ou seja 5 V. Definir um pino digital como **LOW** significa colocar o pino digital em nível lógico baixo (0), ou seja, 0 V.
- Outros conceitos técnicos descritos na Tabela 3 serão detalhados posteriormente nesse material.

Além destas funções, você também pode escrever suas próprias funções, que devem ser escritas fora das funções `setup()` e `loop()`. A criação de uma função deve seguir a sintaxe descrita abaixo:

```
1  Tipo nome_da_funcao(declaração de parâmetros)
2  {
3      Declaração de variável; //opcional
4      Comando 1;
5      Comando 1;
6      ...
7  }
```

Há dois tipos de função: As que não retornam nenhum valor e as que retornam algum valor para a função onde está inserida.

A função que não retornam nenhum valor são do tipo **void**. As funções do tipo **void** não retorna nenhum valor para a função que a chamou, ou seja, as ações executadas nessa função não retornam números, variáveis ou caracteres para a função principal.

Por sua vez, as funções que retornam valor podem ser do tipo **int**, **float**, **string**, etc. Uma função do tipo **int**, por exemplo, retorna um valor inteiro para a função que a chamou. Existem duas formas de retorno de uma função, uma delas é quando o finalizador de função `{}` é encontrado e a outra é usando a declaração **return**.

A declaração **return** termina uma função e retorna um valor desejado.



OBSERVAÇÕES:

- Se a função retorna um valor é obrigatório que seja determinado o tipo de retorno, que pode ser um número inteiro, um caractere ou um número real;

- As variáveis declaradas no parâmetro são variáveis de entrada, cujos tipos também devem ser especificados;
- Barra dupla (//) pode ser utilizada para fazer um breve comentário em alguma linha do código. A função do comentário é deixar o código claro tanto para o programador quanto para outras pessoas. Os comentários são ignorados pelo compilador do código;
- Também é possível comentar várias linhas do código, para isso você deve incluir os comandos /* na linha de início e */ ao final da linha que finaliza o trecho a ser comentado.

BIBLIOTECAS

As bibliotecas são um conjunto de instruções desenvolvidas para executar tarefas específicas relacionadas a um determinado dispositivo. O uso de bibliotecas facilita a conexão a sensores, a uma tela, a um módulo, entre outros, além de poupar tempo do programador.

Algumas bibliotecas já vêm instaladas com o Arduino IDE, outras podem ser incluídas a partir de download e você também pode criar a sua própria. Muitas vezes o fabricante de um sensor, módulos, atuador, entre outros, fornece uma biblioteca para facilitar a programação.

Uma biblioteca padrão é chamada através da seguinte instrução:

```
1 #include <nome_da_biblioteca>
```

Por sua vez, uma biblioteca criada pelo usuário segue a sintaxe:

```
1 #include "nome_da_biblioteca.h"
```



OBSERVAÇÕES:

- Usamos a diretiva `#include` quando desejamos incluir uma biblioteca externa ao nosso Sketch. Com isso, teremos acesso a um grande número de bibliotecas escritas especialmente para a linguagem Arduino;

- Outra diretiva que será utilizada em nossos projetos é a `#define`, que permite dar um nome a um valor constante antes de o programa ser compilado. Uma vantagem da utilização desta diretiva é que as variáveis definidas a partir dela não ocupam espaço na memória de programa do chip;
- Instruções com `#include` e `#define` não são terminadas com `;` (ponto e vírgula).

OPERADORES

OPERADORES ARITMÉTICOS

Os operadores aritméticos são as representações que utilizamos para realizar as operações aritméticas básicas, como somar, subtrair, dividir, multiplicar, entre outras.

Tabela 4 - Operadores aritméticos.

Operador	Nome	Sintaxe	Resultado
+	Adição	$x = y + z$	x é igual a soma de y mais z
-	Subtração	$x = y - z$	x é igual a subtração de y menos z
*	Multiplicação	$x = y * z$	x é igual a multiplicação de y vezes z
/	Divisão	$x = y / z$	x igual a divisão de y por z
%	Resto da divisão	$x = y \% z$	x é igual ao resto da divisão de y por z

OPERADORES DE COMPARAÇÃO E BOOLEANOS

Para programas corretamente a placa microcontroladora UNO é necessário aprender a usar de forma adequada os operadores de comparação e booleanos.

Os operadores de comparação, como o próprio nome diz, compara dois valores retornando verdadeiro (**true**) ou falso (**false**). Observe na Tabela 5 a seguir os operadores de comparação.

Tabela 5 - Operadores de comparação.

Operador	Nome	Sintaxe	Resultado
==	Igual	x == y	Retorna true (verdadeiro) se x for igual a y
!=	Diferente	x != y	Retorna true (verdadeiro) se x for diferente de y
<	Menor que	x < y	Retorna true (verdadeiro) se x for menor que y
>	Maior que	x > y	Retorna true (verdadeiro) se x for maior que y
<=	Menor ou igual a	x <= y	Retorna true (verdadeiro) se x for menor ou igual a y
>=	Maior ou igual a	x >= y	Retorna true (verdadeiro) se x for maior ou igual a y

Os operadores booleanos são utilizados para testes lógicos entre elementos em um teste condicional. Assim como os operadores de comparação, os operadores booleanos também retornam verdadeiro (**true**) e falso (**false**) conforme o resultado dos testes. Os operadores booleanos são:

Tabela 6 - Operadores booleanos.

Operador	Nome
&&	E lógico
	OU lógico
!	NÃO lógico

Operador E lógico

O E lógico resulta em verdadeiro apenas se **ambos** os operandos forem verdadeiros. Representamos o E lógico com **&&**.

Exemplo: Se quisermos verificar se um determinado valor de temperatura se encontra entre uma faixa de valores (entre 30 e 50°C), podemos utilizar:

```
Se temperatura >=30 && temperatura <=50
```

Se o valor encontrado de temperatura for maior ou igual a 30 e menor ou igual a 50 a condição será satisfeita, retornando verdadeiro (`true`).

Operador OU lógico

O OU lógico resulta em verdadeiro se **pelo menos um** dos operadores for verdadeiro. Representamos o OU lógico com `||`.

Exemplo: Se quisermos verificar se o valor de temperatura é igual a pelo menos um valor determinado (30°C ou 50°C), podemos utilizar:

```
Se temperatura == 30 || temperatura == 50
```

Neste caso, se o valor encontrado de temperatura for igual a 30 **ou** igual a 50 a condição será satisfeita, retornando verdadeiro (`true`).

Operador NÃO lógico

O NÃO lógico resulta em verdadeiro se o operando for falso.

Representamos o NÃO lógico com `!`.

Exemplo: Se quisermos mudar o estado de uma variável `x` de verdadeiro para falso e vice-versa, podemos utilizar:

```
!x
```

Neste caso, se `x` for verdadeiro (`true`) o uso do NÃO lógico transformará seu estado para falso (`false`). De mesmo modo, se `x` for falso (`false`) o NÃO lógico transformará seu estado para verdadeiro (`true`).

OPERADORES DE INCREMENTO E DECREMENTO

Os operadores de incremento e decremento são operadores unitários utilizados com a finalidade de adicionar ou subtrair em uma unidade ao valor de uma variável. O operador de incremento é representado por ++ e o de decremento por --.

Os operadores de incremento e decremento podem ser prefixos ou pós-fixos dependendo de serem posicionados antes ou depois do nome da variável a ser implementada ou decrementada.

Tabela 7 - Operadores de incremento e decremento.

Tipo	Operador	Significado
Prefixo	++x	Incrementa x em um e retorna o novo valor de x
	--x	Decrementa x em um e retorna o novo valor de x
Pós-fixos	x++	Incrementa x em um e retorna o valor antigo de x
	x--	Decrementa x em um e retorna o valor antigo de x

Exemplo:

```
1  x=2;
2  y=++x; //x agora contém 3 e y também
3  y=x++; //x contém 4, mas y ainda contém 3
```

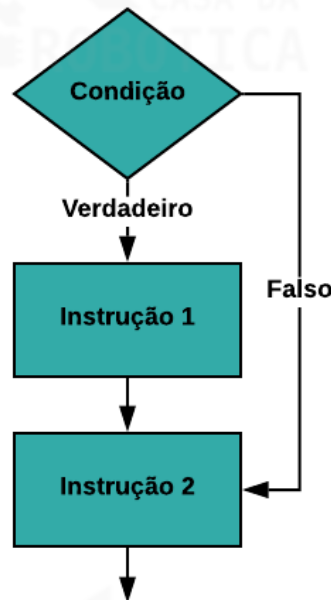
ESTRUTURA DE CONTROLE DE FLUXO

As estruturas de controle são blocos de programação que analisam variáveis e decidem como o programa deve se comportar com base em parâmetros pré-definidos. Em outras palavras, a estrutura de controle de fluxo determina como o programa responderá diante de certas condições ou parâmetros.

A Figura 24 ilustra um exemplo de estrutura de controle, em que dependendo do resultado obtido na condicional (verdadeiro ou falso) o programa executará diferentes instruções.

Neste caso, o resultado da condicional for verdadeiro o programa executará a Instrução 1 e, em sequência, a instrução 2. No entanto, se o resultado da condicional for falso apenas a Instrução 2 será executada pelo programa.

Figura 24 - Estrutura de controle de fluxo.



IF

O comando `if` (se) é uma estrutura de controle de fluxo de seleção. Usamos esse comando para checar uma condição. Se a condição for satisfeita (verdadeiro) uma lista de instrução delimitada por `{ }` (chaves) serão executadas. No entanto, se a condição não for satisfeita (falso) esta lista de instruções não será executada e o programa seguirá a sequência de comandos escritos depois do `if`.

A sintaxe do comando `if` na programação Arduino é:

```
1  if (condição) {
2      Comando 1;
3      Comando 2;
4      ...
5  }
```

IF...ELSE

A combinação dos comandos `if...else` permite maior controle sobre o fluxo de código que o comando `if`, por possibilitar que múltiplos testes sejam agrupados. O comando `else` (senão) será executado se a condição do comando `if` (se) resultar em falso.

A sintaxe dos comandos `if...else` na programação Arduino é:

```
1  if (condição1){  
2      Comando 1;  
3  }  
4  else{  
5      Comando 2;  
6  }
```

Dentro do comando `else` podemos adicionar outro comando `if`, de modo que múltiplos testes podem ser executados ao mesmo tempo. Desta forma, a sintaxe poderá ser a seguinte:

```
1  if (condição1){  
2      Comando 1;  
3  }  
4  else if (condição2){  
5      Comando 2;  
6  }  
7  else{  
8      Comando 3;  
9  }
```

FOR

O comando `for` (para) é uma estrutura de controle de fluxo de repetição. Este comando permite que certo trecho do código seja executado um determinado

número de vezes. O comando `for` é útil para qualquer operação repetitiva e é usado frequentemente com vetores para operar em coleções de dados.

A sintaxe do comando `for` é a seguinte:

```
1  for (inicialização; condição; incremento){
2      Comando 1;
3      ...
4  }
```

A inicialização ocorre primeiro e apenas uma vez. A cada repetição do `loop`, a condição é testada, se verdadeira o bloco de comandos e o incremento são executados. Por sua vez, quando a condição for falsa a repetição termina.

SWITCH...CASE

Da mesma forma que o comando `if`, o comando `switch...case` (selecione...caso) é uma estrutura de controle de fluxo de seleção. Este comando permite especificar códigos diferentes para serem executados em várias condições, funcionando da seguinte maneira: um comando `switch` compara o valor de uma variável aos valores especificados nos comandos `case`. Quando um comando `case` é encontrado, cujo valor é igual ao da variável, o código para esse comando `case` é executado. A sintaxe do comando `switch...case` é a seguinte:

```
1  switch (var){
2      case valor1:
3          comando1;
4          break;
5      case valor2:
6          comando2;
7          break;
8      default:
9          comando3;
10         break;
11 }
```

A variável `var` será comparada ao valor dos vários cases, podendo ser do tipo `int` ou `char`. Os parâmetros `valor1` e `valor2` são constantes do tipo `int` ou `char`.



OBSERVAÇÕES:

- A palavra-chave `break` é utilizada para interromper o comando `switch`, devendo ser escrito ao final de cada case. Sem o comando `break`, o comando `switch` continuará executando as expressões seguintes, até encontrar um `break`.
- A palavra-chave `default` é opcional e é executado quando a variável de comparação do `switch` não corresponde a nenhum valor constante dos cases.

WHILE

O `while` (enquanto) é uma estrutura de controle de fluxo de repetição. As instruções contidas em um `while` serão repetidas continuamente, e infinitamente, até que a sua condicional se torne falsa. Em outras palavras, as instruções contidas no código `while` serão executadas enquanto a condição for satisfeita (verdadeiro).

A forma geral do `while` é:

```
1  while (condição) {  
2      Comando 1;  
3      ...  
4  }
```

DO...WHILE

A estrutura `do...while` (faça...enquanto) funciona de forma semelhante às estruturas `while` e `for`, com exceção de a condição ser testada ao final do `loop`,

de tal modo que as instruções contidas no `do...while` serão executadas pelo menos uma vez.

A sintaxe do comando `do...while` é a seguinte:

```
1  do{  
2      Comando 1;  
3  } while (condição);
```

VAMOS PROGRAMAR!

Até aqui já aprendemos alguns elementos básicos que compõem um Sketch, os operadores e as estruturas de controle de fluxo. Então, vamos começar a programar!

Nos exemplos a seguir, vamos testar o funcionamento da Placa UNO, do Shield Ponte H L298P, dos motores e do módulo seguidor de linha.

ALERTA SONORO! TESTANDO O BUZZER...

Neste projeto, vamos utilizar o buzzer integrado no Shield Ponte H L298P para emitir um alerta sonoro. Apesar de bastante simples, este projeto nos ajudará a compreender sobre a estrutura básica de um programa sequencial desenvolvido no Arduino IDE e como escrever na porta digital da placa UNO. Além disso, servirá para o teste de funcionamento da placa UNO e do Shield Ponte H L298P.

O buzzer é um componente eletrônico que converte um sinal elétrico em onda sonora. Este dispositivo é utilizado para sinalização sonora, sendo aplicado em computadores, despertadores, carros, entre outros.

Elaborando o código

O buzzer integrado no Shield Ponte H L298P será ativado quando colocarmos a porta digital 4 da placa UNO em nível alto (HIGH). Neste projeto, vamos programar a placa UNO para acionar o buzzer em intervalos de 2 segundos.

Para melhor compreensão o código será explicado passo-a-passo a seguir. Neste momento, observe o código abaixo e aproveite para analisar sua estrutura.

```
// Alerta Sonoro! Testando...
int buzzerPin = 4; //Atribui o valor 4 a variável inteira buzzerPin, que
irá representar o pino digital 4

void setup() {
  pinMode(buzzerPin, OUTPUT); //Define ledPin (pino 4) como saída
}

void loop() {
  digitalWrite(buzzerPin, HIGH); //Coloca buzzerPin em nível alto (5V)
  delay(2000); //Espera 2000 milissegundos (2 segundo)
  digitalWrite(buzzerPin, LOW); //Coloca buzzerPin em nível baixo (0V)
  delay(2000); //Espera 2000 milissegundos (2 segundo)
}
```

Você pode simplesmente copiar este código no Arduino IDE, mas vai ser muito mais proveitoso se você escrever o seu próprio código. Ao elaborar o código observe os seguintes pontos:

1- Comentário

A primeira linha do código `// Alerta Sonoro! Testando...` trata-se apenas de um comentário que será ignorado pelo compilador;

2- Declaração da variável

A instrução `int buzzerPin = 4;` atribui o valor 4 a variável inteira `buzzerPin`, que será utilizada para representar a porta digital 4 da placa UNO.

As variáveis são utilizadas para armazenar dados. Em nosso exemplo, a variável é do tipo `int`, ou inteiro. A vantagem da utilização da variável é que se você decidir utilizar outro pino, não será necessário alterar o código em vários locais, basta alterar o valor da variável;

3- Estruturação do código

Conforme vimos anteriormente, a estrutura do código feito no Arduino IDE é composta por duas funções obrigatórias, que são `setup()` e `loop()`, sem elas o Sketch não funcionará.

4- Função de configuração `setup`

Na função `setup()`, vamos escrever as configurações do programa. Para isto, vamos fazer uso da função `pinMode()`, responsável por configurar o modo como um pino especificado irá funcionar, podendo ser como saída ou entrada. Em nosso exemplo, o buzzer embutido na porta Digital 4 (`buzzerPin`) deve ser configurado como porta de saída (`OUTPUT`).



OBSERVAÇÕES:

- Configuramos como saída (`OUTPUT`) todos os dispositivos que desejamos controlar, como: LEDs, buzzer, motores, displays, relés, entre outros;
- Configuramos como entrada (`INPUT`) todos os dispositivos que desejamos receber dados, como: LDR, botões, sensores infravermelhos, sensores ultrassônicos, termistores, reed switch, entre outros.

5- Função principal `loop`

A função `loop()` é a função principal do programa e será executada continuamente enquanto a placa UNO estiver ligada. Em nosso exemplo, desejamos que o buzzer toque em intervalos de 2 segundos e repita continuamente o processo. Desta forma, estas informações devem ser escritas dentro da função `loop()`.

A primeira instrução do `loop()` deve colocar a porta digital 4 em nível lógico alto (`HIGH`). Para isto, utilizamos a função `digitalWrite(buzzerPin, HIGH)`, que escreve um valor `HIGH` para a porta Digital 4. Definir um pino como `HIGH` significa que estamos colocando o pino em nível lógico 1, enviando 5 V para que o buzzer seja ligado. Ao contrário, quando definimos um pino como `LOW` significa que estamos colocando o pino em nível lógico 0, enviando 0 V ou conectado ao terra.

A próxima instrução escrita foi a função `delay(2000)`. Esse comando diz ao microcontrolador para esperar um intervalo 2000 milissegundos, equivalente a 2 segundo, antes de executar a instrução seguinte.

Em seguida, a função `digitalWrite(buzzerPin, LOW)` é utilizada para apagar o buzzer embutido na porta Digital 4. Então, outra instrução de espera por mais 2000 milissegundos é enviada, finalizando a função `loop()`.

Realizando o upload do código

Para carregar o código na placa UNO é necessário configurar a placa e a porta de comunicação, conforme Figura 11 e Figura 12. Em seguida, clique no ícone Upload, destacado na Figura 25.

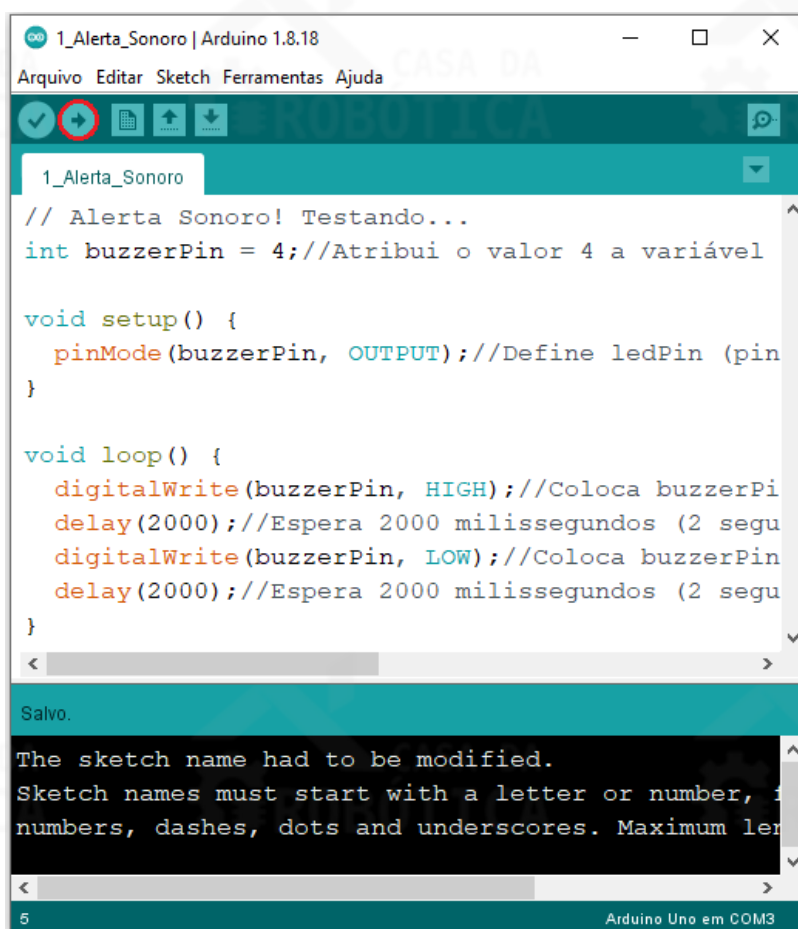
A transferência do código demorará alguns segundos, mas, logo em seguida, o buzzer do Shield Ponte H L298P começará a tocar em intervalos de 2 segundos.



OBSERVAÇÕES:

No CDR CAR, o buzzer do Shield Ponte H L298P pode ser utilizado para sinalizar quando o robô saiu completamente da linha ou quando alcançar a linha de chegada.

Figura 25 - Realizando o upload do código.



MOVIMENTOS INICIAIS! TESTANDO OS MOTORES...

Vamos colocar o nosso CDR CAR para andar? Neste projeto, vamos testar o funcionamento dos motores nosso robô. Os projetos a seguir nos ajudará a compreender melhor o funcionamento do Shield Ponte H 298P e dos motores e serão a base para a construção do nosso seguidor de linha.

MOVENDO O ROBÔ PARA FRENTE

O primeiro teste que vamos realizar será de movimentação do CDR CAR para frente. Para tal, precisamos que os dois motores se movimentem simultaneamente para frente.

Como vimos anteriormente, o sentido de rotação do motor DC é determinado pelo sentido da corrente elétrica que circula em seus enrolamentos e para alterar o sentido de rotação do motor é necessário mudar o sentido da corrente elétrica que o alimenta. Estamos usando o Shield Ponte H L298 para alterar a polaridade da tensão de entrada do motor DC.

Os pinos ENA (pino digital 12) e ENB (pino digital 13) são responsáveis pela direção dos motores. Quando colocados em nível lógico alto, os motores girarão em um sentido. Para inverter a rotação do giro dos motores, basta definirmos estes pinos como nível baixo.

A velocidade dos motores será controlada através dos pinos PWMA (pino digital 10) e PWMB (pino digital 11). Para acionar o PWM nos pinos 10 e 11 da placa UNO, vamos usar a função `analogWrite()`, cuja sintaxe é:

```
analogWrite(pino, valor);
```

em que,

pino: é o pino PWM da placa UNO;

valor: é o ciclo de trabalho (duty cycle) entre 0 (sempre desligado) e 255 (sempre ligado).

O ciclo de trabalho do PWM pode assumir qualquer valor entre 0 e 255. Quando o ciclo de trabalho do PWM do motor for igual a 0, o motor estará parado.

Por sua vez, quando o ciclo de trabalho for igual a 255, o motor girará com velocidade máxima. Quando o ciclo de trabalho for de 127 (duty cycle de 50%), a velocidade do motor será reduzida pela metade.

ATENÇÃO!

Ao realizar os testes de motores, coloque o CDR CAR em cima de um objeto ou plataforma que impeça que suas rodas toquem a mesa/chão, criando uma espécie de macaco. Durante a realização destes testes, o carrinho poderá se mover de forma inesperada e não queremos que seu carrinho caia ou que você precise correr atrás dele 😂.

Elaborando o código

Para movimentar os dois motores simultaneamente para frente, vamos colocar os pinos ENA (pino digital 12) e ENB (pino digital 13) em nível lógico alto (HIGH) e para melhor visualiza a rotação das rodas, vamos definir o ciclo de trabalho de 127 nos pinos PWMA (pino digital 10) e PWMB (pino digital 11), de modo que os motores girem com velocidade média.

Para melhor compreensão o código será explicado passo-a-passo a seguir. Neste momento, observe o código abaixo e aproveite para analisar sua estrutura.

```
// Movimentos iniciais! Testando os motores - Frente...

int motorA_PWM = 10; //Controle de velocidade do motor A (Esquerdo)
int motorB_PWM = 11; //Controle de velocidade do motor B (Direito)

int motorA_EN = 12; //Controle de direção do motor A (Esquerdo)
int motorB_EN = 13; //Controle de direção do motor B (Direito)

int velocidade = 127; //Velocidade de rotação do motor de 0 (velocidade mínima) a 255(velocidade máxima)

void setup() {
  //Configura pinos de saída (OUTPUT)
  pinMode (motorA_PWM, OUTPUT);
  pinMode (motorA_EN, OUTPUT);
  pinMode (motorB_PWM, OUTPUT);
  pinMode (motorB_EN, OUTPUT);
}

void loop() {
```

```
digitalWrite(motorA_EN, HIGH); //Motor Esquerdo - HIGH = Frente
digitalWrite(motorB_EN, HIGH); //Motor Direito - HIGH = Frente
analogWrite(motorA_PWM, velocidade); //PWM do motor esquerdo
analogWrite(motorB_PWM, velocidade); //PWM do motor direito
}
```

Para melhor compreensão do código acompanhe os seguintes passos:

1- Comentário

A primeira linha do código `// Movimentos iniciais! Testando os motores - Frente...` trata-se apenas de um comentário que será ignorado pelo compilador;

2- Declaração das variáveis

Iniciamos o programa declarando as variáveis. Utilizamos as variáveis `motorA_PWM` e `motorB_PWM` para representar os pinos de controle de velocidade dos motores A e B, respectivamente.

Em seguida, declaramos as variáveis `motorA_EN` e `motorB_EN` que controlarão a direção dos motores A e B, respectivamente.

Para representar o valor do ciclo de trabalho PWM, incluímos a variável `velocidade`, atribuindo a ela o valor de 127.

3- Configurar os pinos de saída

Na função `setup()`, definimos os pinos de controle do motor como saída (`OUTPUT`).

4- Mover o motor para frente

Para rotacionar os dois motores simultaneamente, colocamos os pinos ENA (pino digital 12) e ENB (pino digital 13) em nível lógico alto (`HIGH`), por meio das instruções:

```
digitalWrite(motorA_EN, HIGH); //Motor Esquerdo - HIGH = Frente
digitalWrite(motorB_EN, HIGH); //Motor Direito - HIGH = Frente
```

Para que os motores se movam com metade de sua velocidade máxima usamos a função `analogWrite()`, passando como parâmetros os pinos PWMA e PWMB, por meio das variáveis `motorA_PWM` e `motorB_PWM`, e a velocidade, através das instruções:

```
analogWrite(motorA_PWM, velocidade); //PWM do motor esquerdo
analogWrite(motorB_PWM, velocidade); //PWM do motor direito
```

Realizando o upload do código



ATENÇÃO!

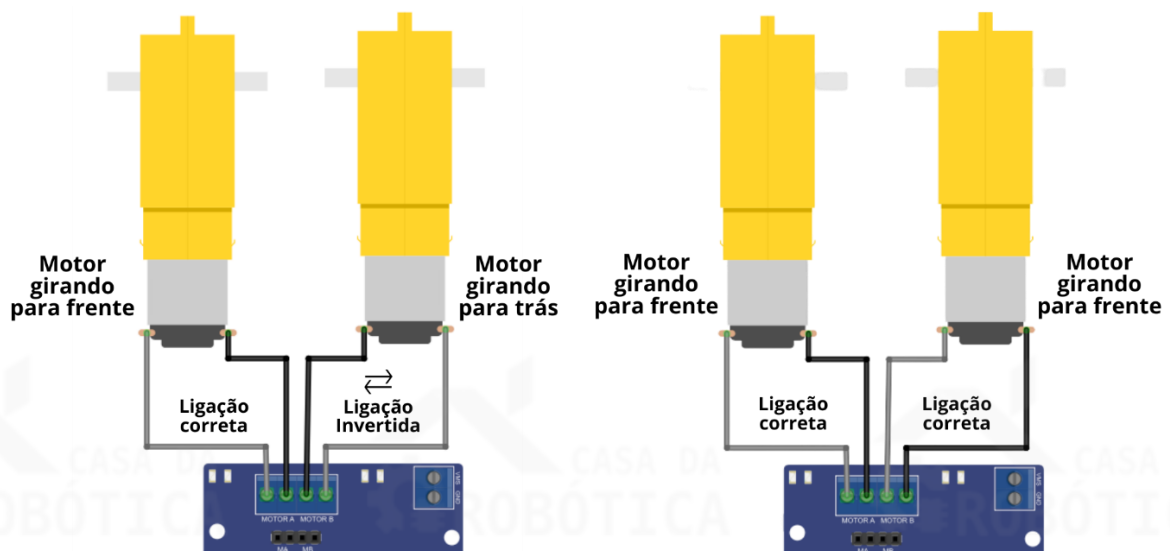
Antes de realizar o upload do código, mude a chave liga/desliga do suporte para 6 pilhas AA para a posição **ON**.

Para carregar o código na placa UNO é necessário configurar a placa e a porta de comunicação, conforme Figura 11 e Figura 12. Em seguida, clique no ícone Upload, destacado na Figura 25.

Após o upload, observe se os dois motores estão girando para frente. Caso alguma das rodas esteja girando para trás ao invés de ir para frente, inverta os fios do motor em questão.

Exemplo: Caso o motor conectado ao Shield Ponte H L298P nos bornes identificados por MOTOR B esteja girando para trás, **remova o cabo USB e mude a chave liga/desliga do suporte para 6 pilhas AA para a posição OFF** e inverta os fios, conforme Figura 26.

Figura 26 - Inverta os fios dos bornes MOTOR B para que o motor gire para frente.



Após isto, ligue novamente o suporte para pilhas 6AA, mantenha o cabo USB desplugado e observe o seu CDR CAR está andando para frente pela primeira vez.

MOVENDO O ROBÔ PARA TRÁS

O segundo teste que vamos realizar será de movimentação do CDR CAR para trás. Para isto, precisamos que os dois motores se movimentem simultaneamente para trás, ao contrário do teste anterior.

Para que os motores se movam para trás, vamos alterar a polaridade da tensão de entrada do motor DC. Como vimos anteriormente, os pinos ENA (pino digital 12) e ENB (pino digital 13) são responsáveis pela direção dos motores e quando colocados em nível baixo, a rotação do giro dos motores será invertida.

Novamente, a velocidade dos motores será controlada através dos pinos PWMA (pino digital 10) e PWMB (pino digital 11).



ATENÇÃO!

Ao realizar os testes de motores, coloque o CDR CAR em cima de um objeto ou plataforma que impeça que suas rodas toquem a mesa/chão, criando uma espécie de macaco. Durante a realização destes testes, o carrinho poderá se mover de forma inesperada e não queremos que seu carrinho caia ou que você precise correr atrás dele 😊.

Elaborando o código

Para movimentar os dois motores simultaneamente para trás, vamos colocar os pinos ENA (pino digital 12) e ENB (pino digital 13) em nível lógico baixo (LOW) e para melhor visualiza a rotação das rodas, vamos definir o ciclo de trabalho de 127 nos pinos PWMA (pino digital 10) e PWMB (pino digital 11), de modo que os motores girem com velocidade média.

Neste momento, observe a semelhança entre o código para mover o robô para frente e o código para mover o robô para trás. Aproveite para analisar suas estruturas. Para melhor compreensão o código será explicado passo-a-passo a seguir.

```
// Movimentos iniciais! Testando os motores - Ré...  
  
int motorA_PWM = 10; //Controle de velocidade do motor A (Esquerdo)  
int motorB_PWM = 11; //Controle de velocidade do motor B (Direito)
```

```

int motorA_EN = 12; //Controle de direção do motor A (Esquerdo)
int motorB_EN = 13; //Controle de direção do motor B (Direito)

int velocidade = 180; //Velocidade de rotação do motor de 0 (velocidade
mínima) a 255(velocidade máxima)

void setup() {
  //Configura pinos de saída (OUTPUT)
  pinMode (motorA_PWM, OUTPUT);
  pinMode (motorA_EN, OUTPUT);
  pinMode (motorB_PWM, OUTPUT);
  pinMode (motorB_EN, OUTPUT);
}

void loop() {
  digitalWrite(motorA_EN, LOW); //Motor Esquerdo - LOW = Ré
  digitalWrite(motorB_EN, LOW); //Motor Direito - LOW = Ré
  analogWrite(motorA_PWM, velocidade); //PWM do motor esquerdo
  analogWrite(motorB_PWM, velocidade); //PWM do motor direito
}

```

Para melhor compreensão do código acompanhe os seguintes passos:

1- Comentário

A primeira linha do código `//Movimentos iniciais! Testando os motores - Ré...` trata-se apenas de um comentário que será ignorado pelo compilador;

2- Declaração das variáveis

Iniciamos o programa declarando as variáveis. Utilizamos as variáveis `motorA_PWM` e `motorB_PWM` para representar os pinos de controle de velocidade dos motores A e B, respectivamente.

Em seguida, declaramos as variáveis `motorA_EN` e `motorB_EN` que controlarão a direção dos motores A e B, respectivamente.

Para representar o valor do ciclo de trabalho PWM, incluímos a variável `velocidade`, atribuindo a ela o valor de 127.

3- Configurar os pinos de saída

Na função `setup()`, definimos os pinos de controle do motor como saída (`OUTPUT`).

4- Mover o motor para frente

Para rotacionar os dois motores simultaneamente para trás, colocamos os pinos ENA (pino digital 12) e ENB (pino digital 13) em nível lógico baixo (**LOW**), por meio das instruções:

```
digitalWrite(motorA_EN, LOW); //Motor Esquerdo - LOW = Ré  
digitalWrite(motorB_EN, LOW); //Motor Direito - HIGH = Ré
```

Para que os motores se movam com metade de sua velocidade máxima usamos a função **analogWrite()**, passando como parâmetros os pinos PWMA e PWMB, por meio das variáveis **motorA_PWM** e **motorB_PWM**, e a velocidade, através das instruções:

```
analogWrite(motorA_PWM, velocidade); //PWM do motor esquerdo  
analogWrite(motorB_PWM, velocidade); //PWM do motor direito
```

Realizando o upload do código



ATENÇÃO!

Antes de realizar o upload do código, mude a chave liga/desliga do suporte para 6 pilhas AA para a posição **ON**.

Para carregar o código na placa UNO é necessário configurar a placa e a porta de comunicação, conforme Figura 11 e Figura 12. Em seguida, clique no ícone Upload, destacado na Figura 25.

Após o upload, observe se os dois motores estão girando para trás simultaneamente. Remova o cabo USB, coloque seu CDR CAR no chão e observe ele está andando para trás pela primeira vez.

DETECTANDO A LINHA! TESTANDO O MÓDULO SEGUIDOR DE LINHA...

Como explicado anteriormente, o módulo seguidor de linha é composto por 3 sensores TCRT5000. O LED emissor de cada um destes sensores emite luz infravermelha continuamente, quando esta luz não for refletida ou não for forte o

suficiente, o sensor terá sua saída digital em nível alto (HIGH) e o LED do sensor correspondente desligará, indicando que a linha foi detectada.

Neste teste, vamos realizar a leitura e imprimir os valores dos sensores TCRT do módulo seguidor de linha no monitor serial do Arduino IDE. Além disso, vamos imprimir qual sensor detectou a linha.



ATENÇÃO!

Use a linha desenhada no fundo do manual de montagem do CDR CAR ou desenhe uma pista em uma superfície branca usando fila isolante preta.

O módulo seguidor de linha possui três saídas de nível lógico, um para cada sensor, indicados por L (esquerdo), C (centro), R (direito). Estas saídas encontram-se conectadas às portas digitais 5, 6 e 7 da placa UNO, respectivamente. Para realizar a leitura destas portas, vamos usar a função `digitalRead()`, cuja sintaxe é:

```
digitalRead(pino);
```

em que,

pino: é o número da porta digital da placa UNO em que se deseja fazer a leitura.

A função `digitalRead()` retorna nível lógico alto (HIGH) ou nível lógico baixo (LOW). A leitura dos sensores TCRT5000 retornarão nível lógico alto (HIGH) quando uma superfície não reflexiva for detectada (linha preta da pista detectada) e nível lógico baixo (LOW) quando estiverem em uma superfície reflexiva (área branca da pista).

Elaborando o código

Para melhor compreensão o código será explicado passo-a-passo a seguir. Neste momento, observe o código abaixo e aproveite para analisar sua estrutura.

```

//Detectando a linha! Testando o módulo seguidor de linha
int sensordireito = 7; //Pino do sensor TCRT direito
int sensorcentro = 6; //Pino do sensor TCRT central
int sensoresquerdo = 5; //Pino do sensor TCRT esquerdo

void setup() {
    Serial.begin(9600); //Inicializa a comunicação serial

    //Configura sensores como entrada (INPUT)
    pinMode(sensordireito, INPUT);
    pinMode(sensorcentro, INPUT);
    pinMode(sensoresquerdo, INPUT);
}

void loop() {
    Serial.print("Sensor Direito:"); //Imprime mensagem "Sensor Direito:"
no monitor serial
    Serial.println(digitalRead(sensordireito)); //Imprime a leitura do
sensor direito no monitor serial
    Serial.print("Sensor Central:"); //Imprime mensagem "Sensor Central:"
no monitor serial
    Serial.println(digitalRead(sensorcentro)); //Imprime a leitura do
sensor central no monitor serial
    Serial.print("Sensor Esquerdo:"); //Imprime mensagem "Sensor Esquerdo:"
no monitor serial
    Serial.println(digitalRead(sensoresquerdo)); //Imprime a leitura do
sensor esquerdo no monitor serial

    if (digitalRead(sensordireito) == 1) { //Se o sensor direito detectar a
linha
        Serial.println("Linha detectada pelo sensor direito");
    }
    if (digitalRead(sensorcentro) == 1) { //Se o sensor central detectar a
linha
        Serial.println("Linha detectada pelo sensor central");
    }
    if (digitalRead(sensoresquerdo) == 1) { //Se o sensor esquerdo detectar
a linha
        Serial.println("Linha detectada pelo sensor esquerdo");
    }

    Serial.println("-----");
    delay(1000);
}

```

Para melhor compreensão do código acompanhe os seguintes passos:

1- Comentário

A primeira linha do código //Detectando a linha! Testando o módulo seguidor de linha trata-se apenas de um comentário que será ignorado pelo compilador;

2- Declaração das variáveis

Para realizar a leitura dos três sensores, declaramos três variáveis:

- A variável `sensordireito` guardará as leituras do sensor indicado por R (sensor direito do módulo) conectado à porta digital 7 da placa UNO;
- A variável `sensorcentro` receberá as leituras do sensor indicado por C (sensor central do módulo) conectado ao pino digital 6 da placa UNO;
- A variável `sensoresquerdo` será responsável por armazenar as leituras do sensor indicado por L (sensor esquerdo do módulo) conectado à porta digital 5 da placa UNO.

3- Inicialização da comunicação serial e configuração dos pinos de entrada

Através da comunicação serial é possível obter os dados que a placa UNO está gerando ou recebendo. Para ter acesso a esses dados e visualizá-los na tela do computador precisamos inicializar a comunicação serial por meio da função `Serial.begin(velocidade);`, em que velocidade é a taxa de transferência em bits por segundo.

A placa UNO consegue emitir dados nas seguintes taxas: 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, ou 115200 bits por segundo. Alguns equipamentos apenas receberão dados caso a placa UNO esteja configurada em uma taxa de transmissão específica, como é o caso do computador que é configurado para receber dados da USB numa velocidade de 9600.

Como receberemos informações (leitura de dados) dos sensores do módulo seguidor de linha, configuramos as variáveis `sensordireito`, `sensorcentro` e `sensoresquerdo` como entrada (`INPUT`).

4- Realizar a leitura digital dos sensores e imprimir no monitor serial

Para leitura dos sensores usamos a função `digitalRead()`, passando como parâmetro as variáveis `sensordireito`, `sensorcentro` e `sensoresquerdo` correspondentes pinos 7, 6 e 5 da placa UNO, respectivamente.

Para imprimir estes valores usamos a função `Serial.println();`, que possibilita a impressão de qualquer tipo de dado no monitor serial do Arduino IDE.



ATENÇÃO!

Para imprimir mensagens no monitor serial é necessário o usar de aspas duplas no início e fim dela.

Exemplo: `Serial.println("Sensor Direito:");`

5- Realizar teste para saber qual sensor detectou a linha

Para identificar qual sensor detectou a linha preta, realizamos testes usando a estrutura de controle de fluxo `if` (Se). Se a leitura do sensor direito retornou nível lógico alto (`HIGH` ou 1) significa que a linha foi detectada e uma mensagem será impressa no monitor serial, informando ao usuário qual foi o sensor.

Fizemos 3 testes, um para cada sensor conforme pode ser observado no seguinte trecho do código:

```
if (digitalRead(sensordireito) == 1) { //Se o sensor direito detectar a
linha
    Serial.println("Linha detectada pelo sensor direito");
}
if (digitalRead(sensorcentro) == 1) { //Se o sensor central detectar a
linha
    Serial.println("Linha detectada pelo sensor central");
}
if (digitalRead(sensoresquerdo) == 1) { //Se o sensor esquerdo detectar
a linha
    Serial.println("Linha detectada pelo sensor esquerdo");
}
```

Realizando o upload do código



ATENÇÃO!

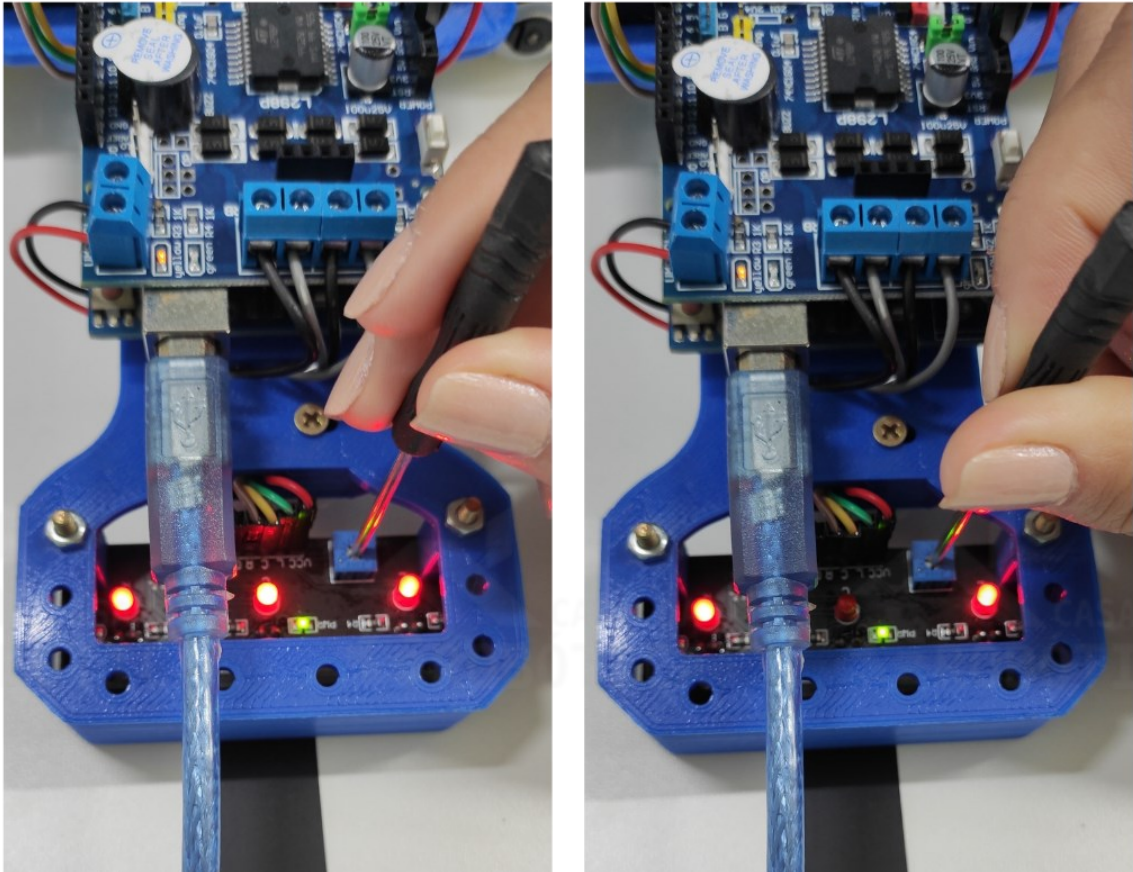
Antes de realizar o upload do código, mude a chave liga/desliga do suporte para 6 pilhas AA para a posição **ON**.

Para carregar o código na placa UNO é necessário configurar a placa e a porta de comunicação, conforme Figura 11 e Figura 12. Em seguida, clique no ícone Upload, destacado na Figura 25.

Após o upload, observe se todos LEDs do módulo seguidor de linha se acender quando o CDR CAR é disposto em uma superfície reflexiva (área branca da pista).

Quando um dos sensores for disposto sobre a linha preta, o LED correspondente a ele será desligado, indicando a detecção da linha. Caso isto não aconteça, ajuste a no potenciômetro a sensibilidade do sensor, conforme detalha a Figura 27.

Figura 27 - Ajuste da sensibilidade dos sensores do módulo seguidor de linha.

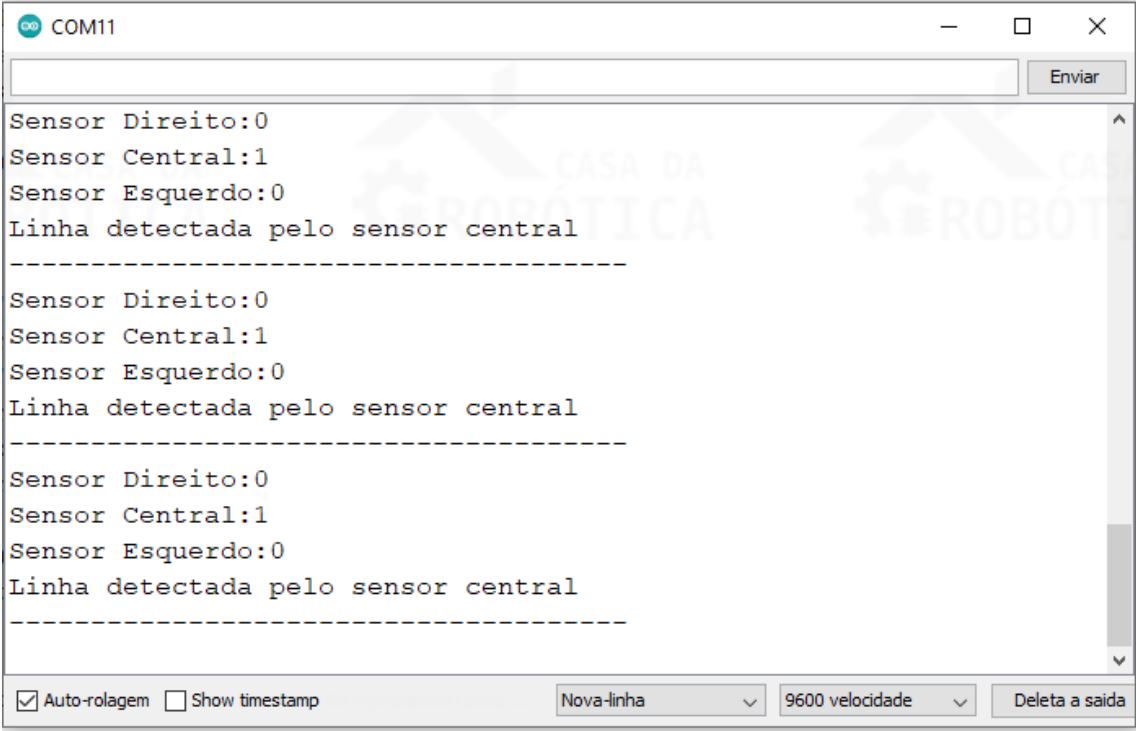


Verificando as leituras no monitor serial

Para visualizar as leituras dos sensores basta abrir o monitor serial do Arduino IDE, clicando ícone  que se encontra na Toolbar. Os dados impressos no monitor serial serão atualizados em intervalos de 1 segundo.

A Figura 28 exibe como o monitor serial do Arduino IDE exibirá as leituras dos sensores do módulo seguidor de linha. Observe que a velocidade de comunicação dos dados deve ser de 9600.

Figura 28 - Monitor serial exibindo as leituras dos sensores do módulo seguidor de linha.



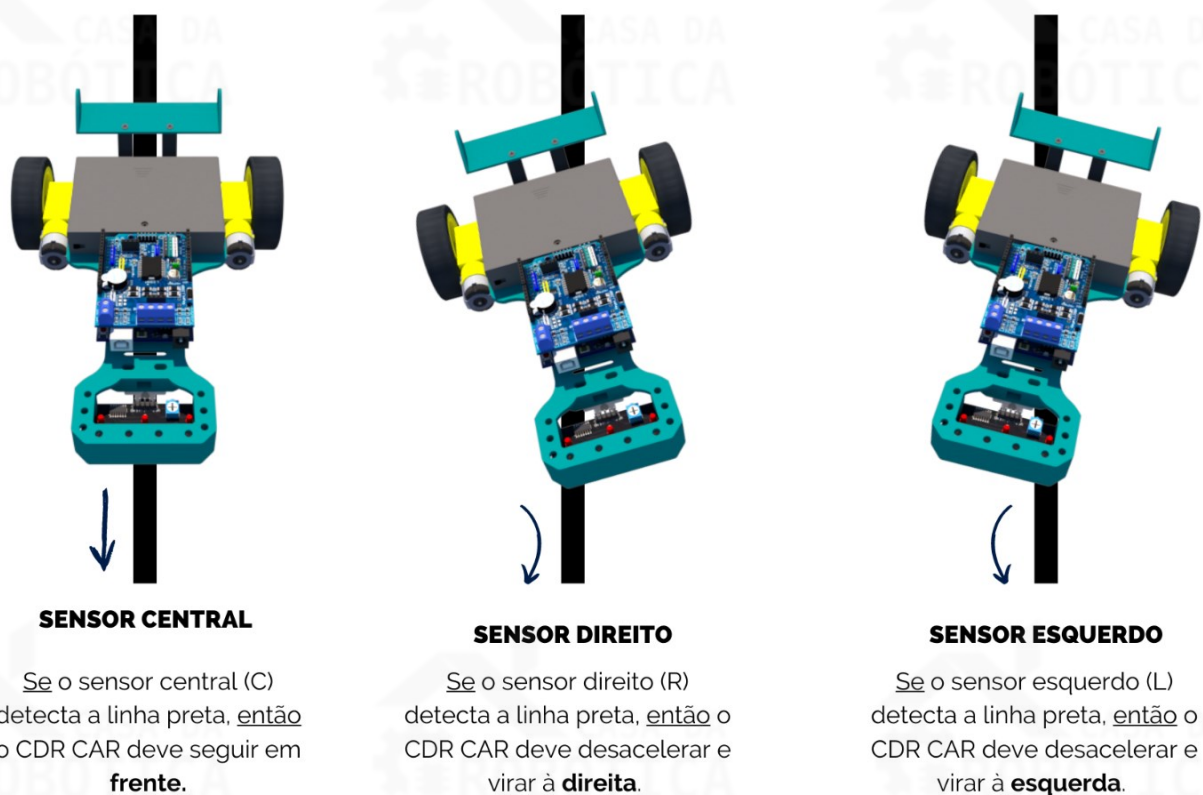
5. ROBÔ SEGUIDOR DE LINHA

CDR CAR: SEGUIDOR DE LINHA

Agora que já montamos o CDR CAR e já testamos seus componentes, vamos programá-lo como seguidor de linha.

O nosso propósito nesse projeto é programar o CDR CAR como seguidor de linha. A identificação da linha preta será efetuada pelos sensores TCRT500 do módulo seguidor de linha. A lógica para manter o CDR CAR seguindo a linha é simples e pode ser observada na Figura 29. Observe que o objetivo principal é sempre manter o sensor central (C) sobre a linha preta.

Figura 29 - Lógica para manter o CDR CAR percorrendo a linha preta.



Com base na Figura 29, verificamos que **3 condições** deverão ser testadas:

- **Se** o sensor central estiver detectando a linha preta (nível lógico alto), **então** o CDR CAR deve seguir em frente;
- **Se** o sensor direito estiver detectando a linha preta (nível lógico alto), **então** o CDR CAR deve desacelerar e virar à direita (até que o sensor central detecte a linha novamente);
- **Se** o sensor esquerdo estiver detectando a linha preta (nível lógico alto), **então** o CDR CAR deve desacelerar e virar à esquerda (até que o sensor central detecte a linha novamente).

Para testar estas três condições, vamos usar a estrutura de controle de fluxo `if` (Se).

Nos exemplos de teste, aprendemos como movimentar o CDR CAR para frente e para trás, e você deve estar se perguntando: Como programamos o seguidor de linha para se mover para direita ou esquerda?

Para que o CDR CAR vá para a **direita**, precisamos acionar o motor A (Esquerdo) e parar o motor B (Direito). Ao contrário, para mover o CDR para a **esquerda**, vamos parar o motor A (Esquerdo) e acionar o motor B (Direito).

A Tabela 8 lista os parâmetros que usaremos para mover o CDR CAR no modo seguidor de linha.

Tabela 8 – Parâmetros para movimentação do CDR CAR.

Direção	Motores	EN	PWM
Frente	Motor A	HIGH	190
	Motor B	HIGH	190
Direita	Motor A	HIGH	170
	Motor B	LOW	0
Esquerda	Motor A	LOW	0
	Motor B	HIGH	170



ATENÇÃO!

Coloque o CDR CAR em cima de um objeto ou plataforma que impeça que suas rodas toquem a mesa/chão, criando uma espécie de macaco. Durante a realização destes testes, o carrinho poderá se mover de forma inesperada e não queremos que seu carrinho caia ou que você precise correr atrás dele 😂.

ELABORANDO O CÓDIGO

Para melhor compreensão o código será explicado passo-a-passo a seguir. Neste momento, observe o código abaixo e aproveite para analisar sua estrutura.

```
//Motores - Saídas digitais
int motorA_PWM = 10; //Controle de velocidade do motor A (Esquerdo)
int motorB_PWM = 11; //Controle de velocidade do motor B (Direito)

int motorA_EN = 12; //Controle de direção do motor A (Esquerdo)
int motorB_EN = 13; //Controle de direção do motor B (Direito)

//Sensor TCRT5000 - Entradas digitais
int sensorDireito = 7; //Pino do sensor TCRT direito
int sensorCentro = 6; //Pino do sensor TCRT central
int sensorEsquerdo = 5; //Pino do sensor TCRT esquerdo

//Variáveis para armazenar a leitura dos sensores TCRT5000
int leituraDireito; //Pino do sensor TCRT direito
int leituraCentro; //Pino do sensor TCRT central
int leituraEsquerdo; //Pino do sensor TCRT esquerdo

//Velocidade PWM
int veloc0 = 0; //Velocidade para parar os motores

int veloc1 = 190; //Velocidade motor A
int veloc2 = 190; //Velocidade motor B

int veloc3 = 170; //Velocidade motor A
int veloc4 = 170; //Velocidade motor B
```

```
void setup() {
    //Configura os motores como saída
    pinMode (motorA_PWM, OUTPUT);
    pinMode (motorA_EN, OUTPUT);
    pinMode (motorB_PWM, OUTPUT);
    pinMode (motorB_EN, OUTPUT);

    //Configura os sensores como entrada
    pinMode (sensorDireito, INPUT);
    pinMode (sensorCentro, INPUT);
    pinMode (sensorEsquerdo, INPUT);
}

void loop() {
    leitura(); //Chama a função de leitura dos sensores
    seguidor(); //Chama a função seguidor de linha
}

void leitura() {
    leituraDireito = digitalRead (sensorDireito); //Leitura do sensor
    direito
    leituraCentro = digitalRead (sensorCentro); //Leitura do sensor central
    leituraEsquerdo = digitalRead (sensorEsquerdo); //Leitura do sensor
    esquerdo
}

void seguidor() {
    if ((leituraDireito == 0) && (leituraCentro == 1) && (leituraEsquerdo
    == 0)) { //Sensor central detectou a linha
        frente(); //CDR CAR move para frente
    }
    if ((leituraDireito == 1) && (leituraCentro == 0) && (leituraEsquerdo
    == 0)) { //Sensor direito detectou a linha
        direita(); //CDR CAR move para a direita
    }
    if ((leituraDireito == 0) && (leituraCentro == 0) && (leituraEsquerdo
    == 1)) { //Sensor esquerdo detectou a linha
        esquerda(); //CDR CAR move para a esquerda
    }
}
}
```

```

void frente() {
    digitalWrite(motorA_EN, HIGH); //Motor A. HIGH = Frente
    digitalWrite(motorB_EN, HIGH); //Motor B. HIGH = Frente

    analogWrite(motorA_PWM, veloc1); //Motor A com pwm = veloc1
    analogWrite(motorB_PWM, veloc2); //Motor B com pwm = veloc2
}

void direita() {
    digitalWrite(motorA_EN, HIGH); //Motor A. HIGH = Frente
    digitalWrite(motorB_EN, LOW); //Motor B. HIGH = Frente

    analogWrite(motorA_PWM, veloc3); //Motor A com pwm = veloc3
    analogWrite(motorB_PWM, veloc0); //Motor B com pwm = veloc0
}

void esquerda() {
    digitalWrite(motorA_EN, LOW); //Motor A. HIGH = Frente
    digitalWrite(motorB_EN, HIGH); //Motor B. HIGH = Frente

    analogWrite(motorA_PWM, veloc0); //Motor A com pwm = veloc0
    analogWrite(motorB_PWM, veloc4); //Motor B com pwm = veloc4
}

```

Para melhor compreensão do código acompanhe os seguintes passos:

1- Declaração das variáveis

Iniciamos o programa declarando as variáveis. Utilizamos as variáveis `motorA_PWM` e `motorB_PWM` para representar os pinos de controle de velocidade dos motores A e B (pinos digitais 10 e 11), respectivamente. Em seguida, declaramos as variáveis `motorA_EN` e `motorB_EN` que controlarão a direção dos motores A e B (pinos digitais 12 e 13), respectivamente.

Logo após, incluímos as variáveis que representam os pinos em que os sensores TCRT5000 do módulo seguidor de linha estão conectados, são elas:

- `sensorDireito`, correspondente ao pino digital 7 da placa UNO;
- `sensorCentro`, correspondente ao pino digital 6 da placa UNO;
- `sensorEsquerdo`, correspondente ao pino digital 5 da placa UNO.

Em seguida, declaramos as variáveis responsáveis por guardar a leitura dos três sensores do módulo seguidor de linha:

- A variável `leituraDireito` guardará as leituras do sensor indicado por R (sensor direito do módulo) conectado à porta digital 7 da placa UNO;
- A variável `leituraCentro` receberá as leituras do sensor indicado por C (sensor central do módulo) conectado ao pino digital 6 da placa UNO;
- A variável `leituraEsquerdo` será responsável por armazenar as leituras do sensor indicado por L (sensor esquerdo do módulo) conectado à porta digital 5 da placa UNO.

Incluimos também as variáveis `veloc0`, `veloc1`, `veloc2`, `veloc3` e `veloc4` para representar o valor do ciclo de trabalho PWM.

2- Configurações dos pinos de entrada e saída

Na função `setup()`, definimos os pinos de controle do motor como saída (`OUTPUT`) e os pinos de leitura dos sensores como entrada (`INPUT`).

3- Função principal

Dentro da função `loop()`, apenas chamamos as funções `leitura()` e `seguidor()`, responsáveis pela leitura dos sensores e movimentação do seguidor de linha.

```
void loop() {  
    leitura(); //Chama a função de leitura dos sensores  
    seguidor(); //Chama a função seguidor de linha  
}
```

Função é um conjunto de comandos que realiza uma tarefa ou ação definida. O uso de funções permite o reaproveitamento de código já construído, evita que um trecho do código seja repetido várias vezes, permite a alteração de um trecho de código de forma rápida e facilita a leitura do programa.

No código seguidor de linha do CDR CAR usamos funções para evitar repetições, garantir organização e facilitar a leitura e entendimento do código.

As funções `leitura()` e `seguidor()` serão executadas continuamente, enquanto a placa UNO estiver ligada.

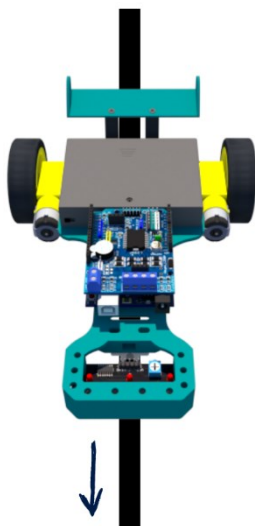
4- Criação da função de `leitura()`

A função `leitura()` contém todas as instruções necessárias para armazenamento da leitura dos três sensores TCRT5000.

Para leitura dos sensores usamos a função `digitalRead()`, passando como parâmetro as variáveis `sensorDireito`, `sensorCentro` e `sensorEsquerdo` correspondentes pinos 7, 6 e 5 da placa UNO. Os resultados destas leituras serão armazenados nas variáveis `leituraDireito`, `leituraCentro` e `leituraEsquerdo`.

5- Criação da função de verificação dos sensores do módulo seguidor de linha

Na função `seguidor()` verificamos qual dos três sensores está detectando a linha e, com base no resultado desta verificação, chamamos a função que fará o CDR CAR se mover para **frente**, **direita** ou **esquerda**.



SENSOR CENTRAL

LEITURA DO SENSOR DIREITO = 0
LEITURA DO SENSOR CENTRAL = 1
LEITURA DO SENSOR ESQUERDO = 0

Se os valores armazenados na variáveis `leituraDireito` e `leituraEsquerdo` forem *iguais* a 0, e o valor de `leituraCentro` for *igual* a 1 (linha detectada), então o CDR CAR deve se **mover para frente**. Deste modo, chamamos a função `frente()`, que será responsável pela movimentação dos motores para frente.

Para verificação desta condição, usamos a estrutura de controle de fluxo `if` (**Se**), o `&&` lógico e *igualdade matemática* (`==`).



ATENÇÃO!

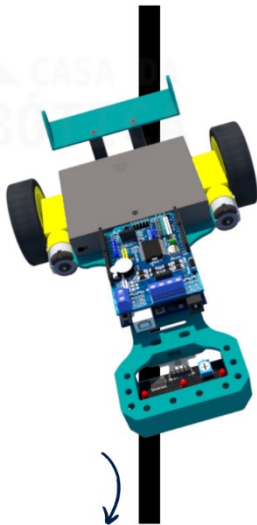
Conforme vimos anteriormente, o E lógico resulta em verdadeiro apenas se **ambos** os operandos forem verdadeiros, sendo representado por &&.

O símbolo = tem o papel exclusivo de atribuição. A igualdade matemática é representada pela dupla igualdade ==.

Transformando a verificação desta condição, em linhas de código obtemos:

```
if ((leituraDireito == 0) && (leituraCentro == 1) && (leituraEsquerdo == 0)) { //Sensor central detectou a linha
    frente(); //CDR CAR move para frente
}
```

Desta maneira, a função `frente()` apenas será executada se `leituraDireito` e `leituraEsquerdo` forem iguais a 0 (zero) e `leituraCentro` for igual a 1



SENSOR DIREITO

LEITURA DO SENSOR DIREITO = 1
LEITURA DO SENSOR CENTRAL = 0
LEITURA DO SENSOR ESQUERDO = 0

Se os valores armazenados na variáveis `leituraCentro` e `leituraEsquerdo` forem iguais a 0, e o valor de `leituraDireito` for *igual* a 1, então o CDR CAR deve **virar para à direita**. Deste modo, chamamos a função `direita()`.

Novamente, usamos a estrutura de controle de fluxo `if (Se)`, o E lógico (&&) e *igualdade matemática* (==).

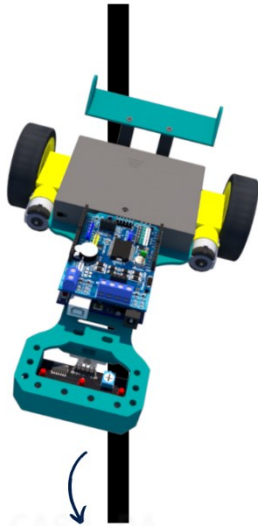
Transformando a verificação desta condição, em linhas de código obtemos:

```

if ((leituraDireito == 1) && (leituraCentro == 0) && (leituraEsquerdo
== 0)) { //Sensor direito detectou a linha
    direita(); //CDR CAR move para a direita
}

```

Assim sendo, a função `direita()` apenas será executada se `leituraCentro` e `leituraEsquerdo` forem iguais a 0 (zero) e `leituraDireito` for igual a 1.



SENSOR ESQUERDO

LEITURA DO SENSOR DIREITO = 0
 LEITURA DO SENSOR CENTRAL = 0
 LEITURA DO SENSOR ESQUERDO = 1

Se os valores armazenados na variáveis `leituraDireito` e `leituraCentro` forem *iguais* a 0, e o valor de `leituraEsquerdo` for *igual* a 1, então o CDR CAR deve **virar para à esquerda**. Deste modo, chamamos a função `esquerda()`.

De mesmo modo, usamos a estrutura de controle de fluxo `if (Se)`, o `E` lógico (`&&`) e *igualdade* matemática (`==`).

Em linhas de código obtemos:

```

if ((leituraDireito == 0) && (leituraCentro == 0) && (leituraEsquerdo
== 1)) { //Sensor esquerdo detectou a linha
    esquerda(); //CDR CAR move para a esquerda
}

```

Portanto, a função `esquerda()` apenas será executada se `leituraDireito` e `leituraCentro` forem iguais a 0 (zero) e `leituraEsquerdo` for igual a 1

6- Criação da função para mover o CDR CAR para frente

Para que o CDR CAR se mova para frente, seus dois motores devem girar simultaneamente para frente. Para isto, colocamos os pinos ENA (pino digital 12) e ENB (pino digital 13) em nível lógico alto (**HIGH**).

Definimos o ciclo de trabalho de 190 nos pinos PWMA (pino digital 10) e PWMB (pino digital 11), de modo que os motores girem com a mesma velocidade. O valor do ciclo de trabalho PWM de 190 foi representado pelas variáveis `veloc1` e `veloc2`.

7- Criação da função para mover o CDR CAR para a direita

Para que o CDR CAR vire à **direita**, precisamos acionar o motor A (Esquerdo) e parar o motor B (Direito). Para tal, colocamos os pinos ENA (pino digital 12) em nível lógico alto (**HIGH**) e ENB (pino digital 13) em nível lógico baixo (**LOW**).

O ciclo de trabalho de 170 foi definido para o pino PWMA (pino digital 10), representado pela variável `veloc3`. Como o motor B (direito) deve ficar parado, definimos o PWMB (pino digital 11) como 0, representado pela variável `veloc0`.

8- Criação da função para mover o CDR CAR para a esquerda

Para mover o CDR se mova à **esquerda**, vamos parar o motor A (Esquerdo) e acionar o motor B (Direito). Para que isto ocorra, colocamos os pinos ENA (pino digital 12) em nível lógico baixo (**LOW**) e ENB (pino digital 13) em nível lógico alto (**HIGH**).

O ciclo de trabalho de 0 foi definido para o pino PWMA (pino digital 10), representado pela variável `veloc0`, uma vez que o motor A deve parar. Definimos o ciclo de trabalho de 170 para o pino PWMB (pino digital 11), representado pela variável `veloc4`.

REALIZANDO O UPLOAD DO CÓDIGO



ATENÇÃO!

Antes de realizar o upload do código, mude a chave liga/desliga do suporte para 6 pilhas AA para a posição **ON**.

Para carregar o código na placa UNO é necessário configurar a placa e a porta de comunicação, conforme Figura 11 e Figura 12. Em seguida, clique no ícone Upload, destacado na Figura 25.

Após o upload, remova o cabo USB e coloque seu CDR CAR na pista, de modo que o sensor TCRT5000 central fique sobre a linha preta. Pronto! Seu CDR CAR agora é um robô seguidor de linha!

CRIANDO A PISTA...

Agora que CDR CAR está pré-programado para detectar e seguir uma linha de forma autônoma, que tal criar uma pista para seu robô?

Para criar a pista, aplique fita isolante sobre uma superfície lisa, clara e grande o suficiente para que o CDR CAR consiga percorrê-la. Esta superfície pode ser o chão, mesa ou cartolina.



IMPORTANTE: Quanto mais clara for a superfície maior será a diferença de reflectância entre ela e a fita preta, facilitando a detecção da linha pelos sensores do módulo seguidor de linha.

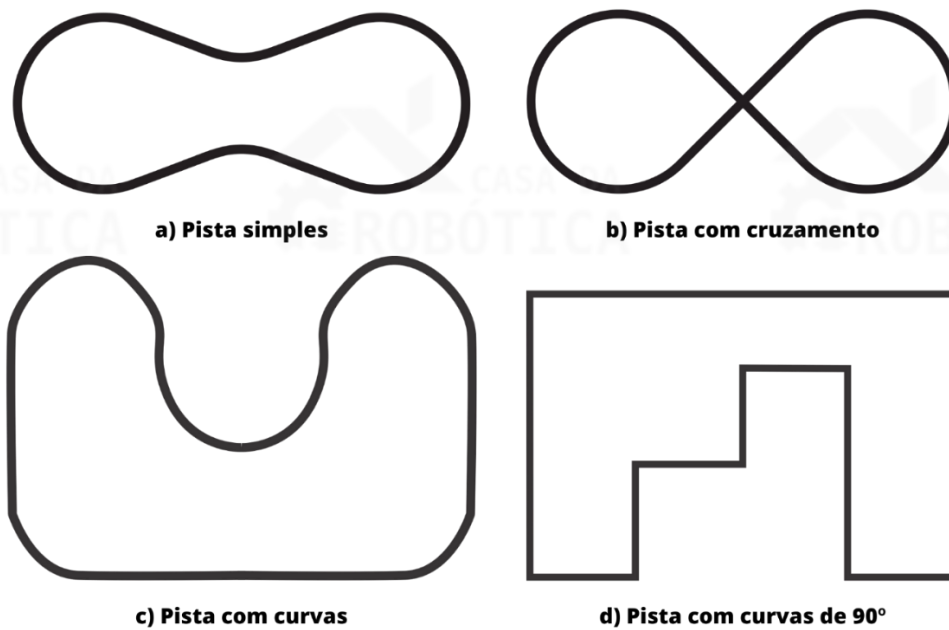


ATENÇÃO: Caso a superfície escolhida tenha sido a mesa, fique atento a movimentação do CDR CAR, pois eventualmente ele pode se mover de forma inesperada, sair do percurso e cair.

DESAFIOS!

Crie uma das pistas da Figura 30 e programe o seu CDR CAR para realizar todo o percurso.

Figura 30 - Modelos de pista para robô seguidor de linha.



- a) **Pista simples**: ideal para testes iniciais com o CDR CAR;
- b) **Pista com cruzamento**: exige que uma condição seja criada para o caso de todos os sensores detectarem linha preta simultaneamente;
- c) **Pista com curvas**: ideal para verificar se o seu CDR CAR consegue fazer curvas mais fechadas;
- d) **Pista com curvas de 90°**: exige mudanças de direção de forma mais abrupta.

6. CONSIDERAÇÕES FINAIS

Chegamos ao final do nosso material de apoio do Kit CDR CAR.

Esperamos que este material tenha auxiliado e contribuído no seu aprendizado.

Caso tenha alguma dúvida entre em contato conosco, afinal "a dúvida é o princípio da sabedoria" (Aristóteles).

Caso você tenha encontrado algum problema no material ou possui alguma sugestão, por favor, entre em contato conosco. Sua opinião é muito importante para nós.

contato@casadarobotica.com

Acompanhe as novidades em nossas redes sociais:

Facebook: @casadaroboticaoficial

Instagram: @casadarobotica

Até a próxima,

Equipe Casa da Robótica

7. REFERÊNCIAS

ARDUINO. **Documentação de Referência da Linguagem Arduino**. Disponível em:

<<https://www.arduino.cc/reference/pt/>>. Acesso em: 07 mar. 2022.

ARDUINO. **Downloads Arduino IDE**. Disponível em:

<<https://www.arduino.cc/en/software>>. Acesso em: 07 mar. 2022.

ELECTROPEAK. **Interfacing L298P H-Bridge Motor Driver Shield with Arduino**.

Disponível em: < <https://electropeak.com/learn/interfacing-l298p-h-bridge-motor-driver-shield-with-arduino/>>. Acesso em: 07 mar. 2022.

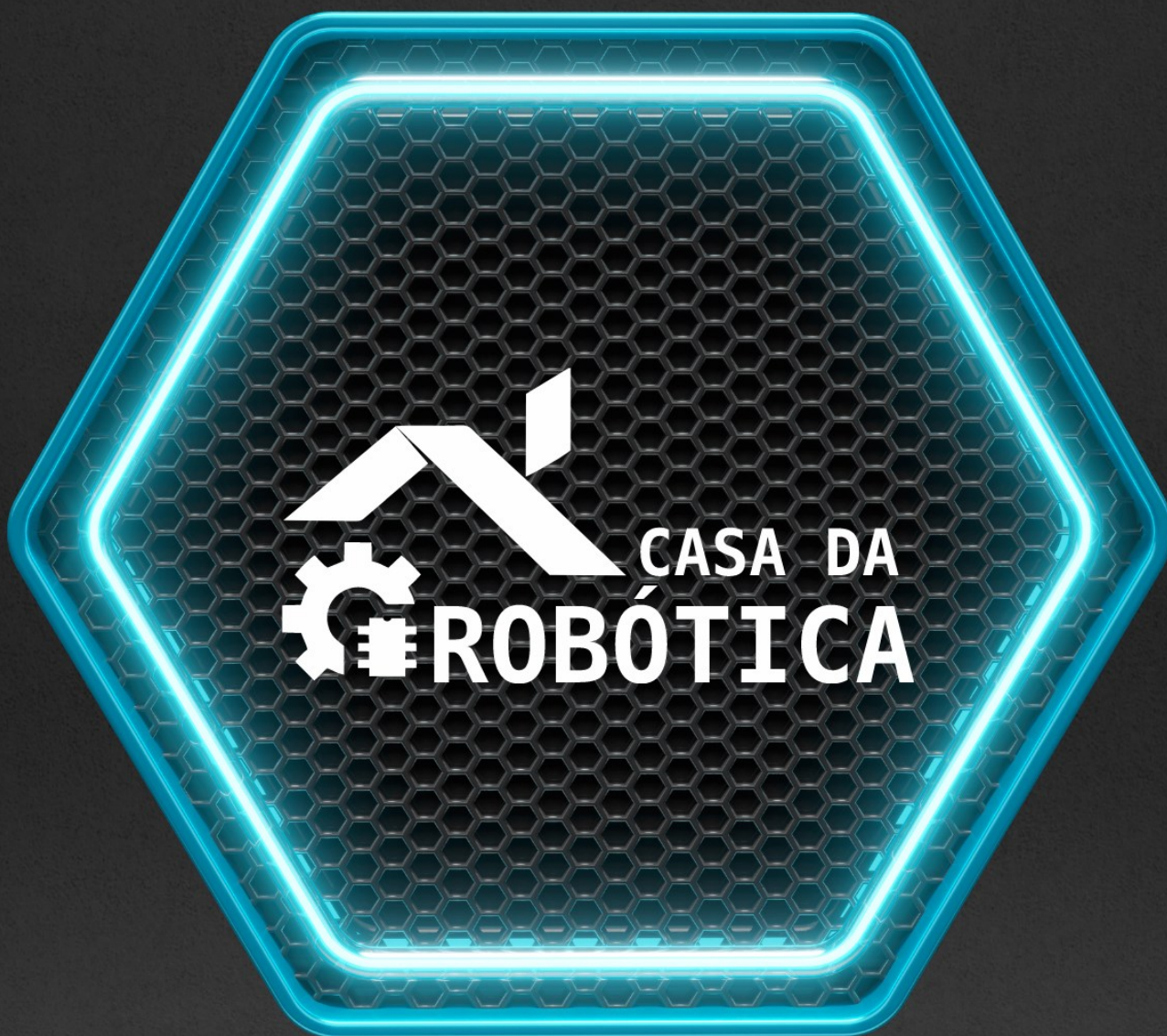
GEFROM. **Monitorizado de AGVs em tempo real em la fábrica de SEAT**.

Disponível em: <<https://www.geprom.com/monitorizado-de-agvs-en-tiempo-real-en-la-fabrica-de-seat-2/>> Acesso em: 07 mar. 2022.

MONK, Simon. **Programação com Arduino: Começando com Sketches**. 2. ed. Bookman, 2017. 180 p.

PACKT. **Robots in Industry**. Disponível em:

<https://subscription.packtpub.com/book/iot_and_hardware/9781789340747/1/ch01lvl1sec05/robots-in-industry>. Acesso em: 07 mar. 2022.



www.casadarobotica.com